

PRACA DYPLOMOWA MAGISTERSKA

Architektura mikroservisów i rozwiązania chmurowe w ekosystemie .NET

*Microservices Architecture and Cloud Solutions in the
.NET Ecosystem*

Mariusz Trzeciak

Nr albumu: 133583

Kierunek: Informatyka

Forma studiów: stacjonarne

Poziom studiów: II

Promotor pracy:

dr inż. Bartosz Kowalczyk

Praca przyjęta dnia:

Podpis promotora:

Częstochowa, 2026

*Niniejszym chciałbym serdecznie podziękować
Promotorowi, dr inż. Bartoszowi Kowalczykowi,
za cenne wskazówki i cierpliwość
w trakcie realizacji niniejszej pracy.*

Spis treści

Wykaz skrótów	7
1 Wstęp	9
1.1 Przegląd badań porównawczych	10
1.2 Cel pracy	12
1.3 Zakres pracy	13
1.4 Struktura pracy	14
2 Wprowadzenie do architektury mikroserwisów w ekosystemie .NET	15
2.1 Definicja i charakterystyka architektury mikroserwisowej	15
2.2 Porównanie z architekturą monolityczną	16
2.3 Ekosystem .NET w kontekście mikroserwisów	17
3 Mechanizmy komunikacji w architekturze mikroserwisowej	19
3.1 Komunikacja synchroniczna	19
3.2 gRPC: architektura i implementacja	20
4 gRPC-Web i komunikacja frontend-backend	23
4.1 Ograniczenia natywnego gRPC w przeglądarkach	23
4.2 Architektura gRPC-Web	25
4.3 Implementacja we frameworku Angular	25
5 Wzorce projektowe i architektoniczne	27
5.1 Wzorzec Cache-Aside	27
5.2 Rola serwera pośredniczącego Envoy	28
6 Zrealizowany system i jego wdrożenie	29
6.1 Architektura badanego systemu testowego	29
6.2 Wdrożenie i eksploatacja w środowisku chmurowym	31
6.3 Obserwowalność systemu	32
7 Metodologia badań i projekt eksperymentu	35
7.1 Cele i hipotezy badawcze	35
7.2 Adaptacja systemu na potrzeby eksperymentu	36

7.3	Metodyka pomiaru	38
7.4	Scenariusze i macierz badawcza	40
7.5	Środowisko testowe	41
7.6	Metryki i sposób agregacji wyników	44
7.7	Ograniczenia metodologiczne	46
8	Porównawcza analiza wydajności REST i gRPC-Web	49
8.1	Wolumen przesyłanych danych	49
8.2	Model liczby wymian pakietów	50
8.3	Analiza narzutu protokołu (Echo)	52
8.4	Analiza wydajności odczytu (Products)	53
8.5	Analiza wydajności zapisu (Orders)	55
8.6	Porównanie konfiguracji komunikacyjnych	57
8.7	Skalowanie z liczbą klientów i granica stanowiska pomiarowego . . .	58
8.8	Powtarzalność pomiarów i przedziały ufności	59
8.9	Weryfikacja hipotez badawczych	61
8.10	Rekomendacje i wytyczne wdrożeniowe	62
9	Podsumowanie	65
	Bibliografia	69
	Spis rysunków	72
	Spis tabel	74
	Spis listingów	75
	Streszczenie	76
	Summary	78
	Słowa kluczowe	80

Wykaz skrótów

Skrót	Rozwinięcie
ACID	Atomicity, Consistency, Isolation, Durability: właściwości transakcji bazodanowych.
AKS	Azure Kubernetes Service.
ALPN	Application-Layer Protocol Negotiation: mechanizm negocjowania protokołu warstwy aplikacyjnej.
API	Application Programming Interface: interfejs programistyczny aplikacji.
BDP	Bandwidth-Delay Product: iloczyn przepustowości łącza i jego opóźnienia.
CDP	Chrome DevTools Protocol.
CI/CD	Continuous Integration / Continuous Delivery: ciągła integracja i ciągłe dostarczanie oprogramowania.
CORS	Cross-Origin Resource Sharing: mechanizm współdzielenia zasobów między źródłami.
CPU	Central Processing Unit: jednostka centralna procesora.
DDD	Domain-Driven Design: projektowanie sterowane dziedziną.
DOM	Document Object Model: model obiektowy dokumentu.
gRPC	Framework zdalnego wywoływania procedur.
HATEOAS	Hypermedia As The Engine Of Application State.
HPACK	Header Compression for HTTP/2: algorytm kompresji nagłówków HTTP/2.
HTTP	Hypertext Transfer Protocol: protokół przesyłania hipertekstu.
JSON	JavaScript Object Notation: tekstowy format wymiany danych.
ORM	Object-Relational Mapping: mapowanie obiektowo-relacyjne.
PS	Page Size: liczba rekordów na stronie wyników.
RAM	Random Access Memory: pamięć operacyjna.
REST	Representational State Transfer: styl architektoniczny usług sieciowych.
RFC	Request for Comments: dokument normalizacyjny Internet Engineering Task Force.
RPC	Remote Procedure Call: zdalne wywoływanie procedur.
RTT	Round-Trip Time: czas obiegu pakietu między klientem a serwerem.

Skrót	Rozwinięcie
SSE	Server-Sent Events: mechanizm przesyłania zdarzeń z serwera do klienta.
TCP	Transmission Control Protocol: protokół transportowy zapewniający niezawodne dostarczanie danych.
TLS	Transport Layer Security: protokół zabezpieczający komunikację sieciąową.
TTL	Time To Live: czas ważności wpisu w pamięci podręcznej.
URI	Uniform Resource Identifier: identyfikator zasobu.
URL	Uniform Resource Locator: adres zasobu w sieci.
VU	Virtual User: wirtualny użytkownik generujący obciążenie.

Rozdział 1

Wstęp

Współczesne aplikacje internetowe coraz częściej funkcjonują jako systemy rozproszone, w których poszczególne funkcjonalności są realizowane przez niezależne usługi. Jednym z podejść do budowy takich systemów jest architektura mikroserwisowa, pozwalająca na podział aplikacji na mniejsze, niezależne komponenty. Wraz z takim podziałem pojawia się potrzeba sprawnej wymiany danych pomiędzy usługami oraz pomiędzy warstwą backendową a aplikacją kliencką.

W komunikacji z aplikacjami internetowymi szeroko stosowanym rozwiązaniem jest styl architektoniczny REST, którego założenia zostały opisane przez Roya Fieldinga [17]. W praktycznych zastosowaniach dane są często przesyłane w formacie JSON. Alternatywę stanowi gRPC, wykorzystujące binarny format Protocol Buffers oraz protokół HTTP/2 [20]. Rozwiązanie to zostało zaprojektowane z myślą o wydajnej komunikacji pomiędzy usługami, a badania porównawcze wskazują na możliwe korzyści związane między innymi z rozmiarem przesyłanych danych i czasem odpowiedzi [1].

Charakter komunikacji w przeglądarce internetowej powoduje jednak, że natywne gRPC nie może być wykorzystywane w niej w taki sam sposób jak w środowisku serwerowym. Rozwiązaniem tego problemu jest gRPC-Web, umożliwiające wykorzystanie mechanizmów gRPC w aplikacjach działających w przeglądarce [22]. Może ono być obsługiwane między innymi z wykorzystaniem Envoy Proxy [23] lub bezpośrednio przez serwer aplikacyjny, na przykład w środowisku ASP.NET Core [25]. Różne sposoby realizacji tej komunikacji wprowadzają odmienne charakterystyki techniczne, których wpływ na wydajność nie jest oczywisty.

W praktycznym systemie wynik pomiaru zależy również od warunków, w jakich wykonywane są zapytania. Znaczenie może mieć między innymi rozmiar przesyłanych danych, liczba równoczesnych klientów, sposób obsługi pamięci podręcznej oraz konieczność wykonania operacji na bazie danych. Z tego względu porównanie samych mechanizmów serializacji lub pojedynczych zapytań nie pozwala w pełni ocenić zachowania poszczególnych rozwiązań w warunkach zwiększonego obciążenia.

1.1 Przegląd badań porównawczych

Do przeglądu wybrano prace empiryczne porównujące REST z gRPC lub GraphQL. Uwzględniono publikacje, w których podano środowisko wykonania, rodzaj obciążenia oraz co najmniej jedną miarę wydajności, taką jak czas odpowiedzi, rozmiar komunikatu albo wykorzystanie zasobów. Zestawienie służy określeniu, które elementy komunikacji zostały już zbadane i czy uzyskane wyniki można odnieść do aplikacji działającej w przeglądarce.

Wyniki badań REST i gRPC

Wyniki dotychczasowych porównań nie wskazują protokołu, który byłby szybszy w każdym rodzaju operacji. Niswar i współautorzy [1] odnotowali najkrótsze czasy odpowiedzi dla gRPC w mikroserwisach napisanych w Go. Śliwa i Pańczyk [2], badający implementacje .NET 5.0, uzyskali korzystne wyniki REST w wielu scenariuszach, mimo mniejszego wolumenu danych przesyłanego przez gRPC. Z kolei eksperymenty Bolanowskiego i współautorów [3] oraz Kamińskiego i współautorów [4] pokazały, że wynik zmienia się wraz z typem danych, operacją i konfiguracją transportu.

Rozbieżności pomiędzy wynikami można częściowo wyjaśnić konstrukcją eksperymentów. Wraz ze zmianą protokołu zmieniały się zwykle format reprezentacji, wersja HTTP, biblioteka serwerowa, a czasem również sposób dostępu do danych. Mierzono więc kompletne warianty implementacyjne, a nie wyizolowany koszt protokołu. Różnił się także punkt zakończenia pomiaru. JMeter i k6 rejestrują obsługę żądania przez klienta testowego, lecz nie uwzględniają deserializacji w kodzie aplikacji frontendowej ani aktualizacji interfejsu użytkownika.

Badania dotyczące GraphQL pomagają oddzielić wpływ protokołu od wpływu zakresu odpowiedzi. Brito, Mombach i Valente [5] wykazali, że migracja klientów API z REST do GraphQL może znacznie ograniczyć liczbę zwracanych pól i rozmiar odpowiedzi. Jest to korzyść wynikająca z doboru danych przez klienta, a nie z binarnych właściwości reprezentacji. Vadlamani i współautorzy [6] otrzymali wyniki zależne od konkretnego zapytania do API GitHuba. W badaniu Jina i współautorów [7] na wynik wpływała również odmienna infrastruktura wdrożeniowa porównywanych rozwiązań. Publikacje te potwierdzają, że porównywane ścieżki powinny zwracać ten sam zakres informacji i wykonywać tę samą logikę biznesową.

Zakres i kryteria analizy

Przegląd obejmuje publikacje empiryczne, w których opisano środowisko wykonania, rodzaj obciążenia i co najmniej jedną miarę wydajności: czas odpowiedzi, wolumen danych albo wykorzystanie zasobów. Dokumentacje techniczne wykorzystano

wyłącznie do wyjaśnienia mechanizmów implementacyjnych. Nie stanowią one samodzielnego dowodu przewagi wydajnościowej jednego rozwiązania nad drugim.

Przy interpretacji prac rozpatrywano przede wszystkim granicę pomiaru, użyte środowisko, funkcjonalną równoważność operacji oraz sposób dostępu do danych. Generator obciążenia rejestruje czas obsługi wywołania API, podczas gdy pomiar realizowany w przeglądarce obejmuje dodatkowo odebranie odpowiedzi, jej dekodowanie oraz aktualizację interfejsu. Te wartości odpowiadają odmiennym pytaniom badawczym i nie powinny być traktowane jako zamienne.

Porównania REST i gRPC

Śliwa i Pańczyk [2] porównali REST, GraphQL i gRPC w trzech aplikacjach o tej samej funkcjonalności, zbudowanych na platformie .NET 5. W ich testach REST uzyskał korzystne wyniki czasu odpowiedzi i przepustowości w wielu scenariuszach, podczas gdy gRPC przesyłało mniej danych. Pokazuje to, że mniejszy komunikat nie gwarantuje krótszego czasu całej operacji. Badanie jest istotnym punktem odniesienia ze względu na wspólny ekosystem .NET, lecz pomiar generatora obciążenia nie obejmuje aktualizacji DOM.

Bolanowski i współautorzy [3] oraz Kamiński i współautorzy [4] zwracają uwagę na zależność wyników od rodzaju operacji, środowiska uruchomieniowego i konfiguracji transportu. W porównaniach tego rodzaju równocześnie zmieniają się format reprezentacji, wersja HTTP, biblioteka serwerowa, a niekiedy także ścieżka dostępu do danych. Wynik dotyczy więc kompletnego wariantu implementacyjnego, a nie wyizolowanego kosztu pojedynczego protokołu. Wniosek ten ma bezpośrednie znaczenie dla interpretacji eksperymentu przedstawionego w niniejszej pracy.

GraphQL jako odrębny wymiar porównania

GraphQL umożliwia klientowi dobór pól odpowiedzi, przez co może zmieniać zarówno objętość danych, jak i liczbę żądań potrzebnych do realizacji operacji. Jest to inna oś optymalizacji niż zastosowanie zwartego kodowania tej samej reprezentacji. Badanie Brito, Mombach i Valente [5] pokazuje, że ograniczenie zakresu zwracanych pól może istotnie zmniejszyć odpowiedź. Z kolei Vadhvani i współautorzy [6] nie wskazali rozwiązania szybszego dla każdej analizowanej pary zapytań.

Tabela 1.1 zestawia elementy najważniejsze dla przenoszenia wyników do badania aplikacji przeglądarkowej. Zróżnicowanie metryk i warunków nie uzasadnia uśredniania przewag procentowych raportowanych w poszczególnych publikacjach.

Tabela 1.1: Zakres omówionych badań i ograniczenia ich odniesienia do eksperymentu przeglądarkowego

Publikacja	Przedmiot oceny	Granica porównania
Śliwa, Pańczyk [2]	REST, GraphQL i gRPC; .NET, generator obciążenia	Czas API, bez aktualizacji DOM
Niswar i in. [1]	REST, GraphQL i gRPC; Go, Redis, JMeter	Odczyty z cache, inne środowisko wykonania
Bolanowski i in. [3]	REST i gRPC w mikroservisach	Wynik zależny od zadania i konfiguracji
Kamiński i in. [4]	REST, gRPC, GraphQL i WebSocket	Zależność od środowiska i bibliotek
Vadlamani i in. [6]	REST i GraphQL w API GitHuba	Mała liczba zapytań, brak kontroli implementacji

Ograniczenia porównywalności i zakres wkładu pracy

Natywne gRPC oraz gRPC-Web nie są zamienne z perspektywy aplikacji przeglądarkowej. Ograniczenia przeglądarki wymagają odmiennego sposobu realizacji komunikacji [22, 24]. Ponadto REST nie jest ograniczony do HTTP/1.1, a zmiana formatu danych, wersji HTTP, TLS i obecności proxy prowadzi do porównania całych ścieżek komunikacji. Pomiar mniejszej odpowiedzi wspiera wyjaśnienie zaobserwowanej różnicy, ale nie wyznacza samodzielnie udziału każdego czynnika.

W omówionych publikacjach nie przedstawiono łącznego porównania REST, gRPC-Web przez Envoy oraz gRPC-Web obsługiwane bezpośrednio w ASP.NET Core, prowadzonego do momentu aktualizacji DOM i z rozróżnieniem rozmiaru odpowiedzi oraz ścieżek dostępu do danych. Niniejsza praca uzupełnia ten obraz o powtarzany eksperyment w systemie .NET 10 i Angular. Wyniki mają charakter kontekstowy: opisują zachowanie trzech konkretnych konfiguracji, a nie uniwersalny ranking technologii. GraphQL uwzględniono w przeglądzie jako alternatywny sposób doboru danych, lecz nie jako wariant eksperymentalny.

1.2 Cel pracy

Głównym celem pracy jest zaprojektowanie, implementacja oraz eksperymentalna walidacja systemu mikroservisowego w ekosystemie .NET 10, umożliwiającego porównawczą analizę wydajności protokołów komunikacji na granicy pomiędzy aplikacją przeglądarkową a warstwą usług.

W odniesieniu do stanu badań omówionego w sekcji 1.1 cel ten oznacza sprawdzenie, w jakich warunkach korzyści obserwowane w testach API przekładają

się na czas obsługi operacji obejmujący przetwarzanie odpowiedzi przez aplikację kliencką.

Problem badawczy wynika z różnicy pomiędzy warunkami, w których zwykle wykorzystuje się natywne gRPC, a warunkami komunikacji aplikacji internetowej. W przeglądarce stosowany jest wariant gRPC-Web, którego obsługa wymaga warstwy pośredniczącej albo bezpośredniego wsparcia po stronie serwera. Równocześnie na czas operacji widziany przez użytkownika wpływa nie tylko transmisja danych, lecz także dostęp do danych, praca klienta oraz warunki sieciowe. Z tego względu wnioski oparte wyłącznie na czasie odpowiedzi API lub rozmiarze komunikatu nie wystarcza do oceny ścieżki przeglądarka–backend.

Dla osiągnięcia celu zaprojektowano trzy scenariusze testowe: Echo, służący ocenie narzutu komunikacji przy deterministycznej odpowiedzi, Products, obejmujący odczyt katalogu z ciepłą i zimną pamięcią podręczną, oraz Orders, odwierciany operację zapisu. W każdym scenariuszu porównano REST oraz dwa warianty gRPC-Web: ścieżkę przez Envoy Proxy i obsługę bezpośrednio w ASP.NET Core.

Analizie poddano czas od interakcji użytkownika do aktualizacji drzewa dokumentu, wolumen odpowiedzi, liczbę równoległych klientów, rozmiar ładunku oraz stan pamięci podręcznej. Tak określony zakres umożliwia ocenę wyników w odniesieniu do konkretnych konfiguracji systemu, bez przypisywania badanym rozwiązaniom uniwersalnej przewagi wydajnościowej.

Pomiar czasu kończy się po naniesieniu otrzymanej odpowiedzi na drzewo dokumentu, dlatego opisuje operację realizowaną przez aplikację internetową, a nie sam czas obsługi żądania po stronie serwera. Nie obejmuje on fizycznego wyświetlenia kolejnej klatki na ekranie. Rozróżnienie tych granic pomiaru jest konieczne przy interpretacji wyników i porównywaniu ich z badaniami wykorzystującymi generatory żądań.

Praca koncentruje się na odczytach i zapisach o ustalonej semantyce danych. Nie obejmuje porównania GraphQL, ponieważ mechanizm doboru pól przez klienta zmieniałby zakres przetwarzanych danych i wymagał odrębnego projektu scenariuszy. Nie oceniano również komunikacji asynchronicznej ani brokerów wiadomości, gdyż nie były elementem zrealizowanego systemu testowego.

1.3 Zakres pracy

Zakres pracy obejmuje architekturę mikroservisową w ekosystemie .NET 10 oraz technologie wykorzystane do realizacji i porównania komunikacji pomiędzy aplikacją przeglądarkową a usługami: REST, gRPC-Web, Protocol Buffers, ASP.NET Core, Angular, Envoy Proxy, PostgreSQL, Redis, Prometheus i Grafana.

Metodyka badawcza polega na zaprojektowaniu autorskiego systemu

oraz przeprowadzeniu powtarzalnych pomiarów typu *end-to-end* w rzeczywistej przeglądarce, porównujących REST i warianty gRPC-Web przy kontrolowanej zmianie rozmiaru ładunku, stanu pamięci podręcznej, liczby równoległych klientów i scenariusza operacji.

1.4 Struktura pracy

W rozdziale 2 przedstawiono fundamenty architektury mikroservisowej, porównanie z modelem monolitycznym oraz specyfikę ekosystemu .NET. W rozdziale 3 omówiono komunikację synchroniczną oraz architekturę frameworka gRPC. W rozdziale 4 scharakteryzowano ograniczenia natywnego gRPC w przeglądarkach, architekturę protokołu gRPC-Web oraz jego implementację we frameworku Angular. W rozdziale 5 opisano wzorzec Cache-Aside i rolę serwera pośredniczącego Envoy w badanym systemie. W rozdziale 6 przedstawiono architekturę badanego systemu testowego, proces wdrożenia w środowisku chmurowym oraz warstwę obserwowalności. W rozdziale 7 sformułowano hipotezy badawcze, projekt aplikacji testowej, procedury pomiarowe oraz macierz scenariuszy badawczych. W rozdziale 8 przeprowadzono porównawczą analizę wydajności protokołów gRPC-Web i REST oraz weryfikację hipotez. Pracę zamyka podsumowanie zestawiające rezultaty badań, wnioski inżynierskie oraz kierunki dalszego rozwoju zrealizowanej platformy.

Rozdział 2

Wprowadzenie do architektury mikroserwisów w ekosystemie .NET

Przedmiotem rozdziału jest architektura mikroserwisów w kontekście nowoczesnego wytwarzania oprogramowania, z uwzględnieniem ekosystemu platformy .NET. Przedstawiono w nim podstawowe cechy architektury monolitycznej i mikroserwisowej, ich najważniejsze zalety i ograniczenia oraz kluczowe pojęcia i zasady projektowe związane z budową systemów mikroserwisowych.

2.1 Definicja i charakterystyka architektury mikroserwisowej

Architektura mikroserwisowa (ang. *microservices architecture*) polega na budowie systemu informatycznego jako zbioru niewielkich, autonomicznych usług realizujących ściśle określone funkcje biznesowe. W ujęciu Fowlera i Lewisa mikroserwisy stanowią niezależnie wdrażalne jednostki, uruchamiane w odrębnych procesach i komunikujące się za pomocą lekkich mechanizmów sieciowych [9]. Organizacja usług wokół zdolności biznesowych (ang. *business capabilities*) sprawia, że ich granice wyznacza zakres odpowiedzialności dziedzinowej, a nie podział na warstwy techniczne. Umożliwia to niezależne skalowanie, wdrażanie oraz modyfikowanie poszczególnych komponentów.

Nadrzędną cechą definiującą mikroserwis jest jego *niezależna wdrażalność* (ang. *independent deployability*), czyli możliwość wdrożenia pojedynczej usługi na środowisko produkcyjne bez konieczności jednoczesnej aktualizacji pozostałych elementów systemu [8]. Warunkiem koniecznym do jej osiągnięcia jest zachowanie luźnego sprzężenia, polegającego na minimalizacji wiedzy o wewnętrznej implementacji innych usług. Do najczęstszych przyczyn utraty tej autonomii należą współdzielone biblioteki zawierające logikę biznesową oraz bezpośredni dostęp do bazy danych innego mikroserwisu. Oba te przypadki wprowadzają ukryte

zależności, które w praktyce wymuszają zsynchronizowane wdrożenia.

Wymiana danych pomiędzy usługami odbywa się za pomocą lekkich protokołów komunikacyjnych, w tym HTTP oraz systemów kolejkowych. W odniesieniu do protokołu HTTP stopień dojrzałości implementacji stylu REST opisuje model dojrzałości Richardsona (ang. *Richardson Maturity Model*), klasyfikujący usługi według wykorzystania zasobów, standardowych metod HTTP oraz mechanizmów hipermediaalnych (HATEOAS) [18]. Model ten nie definiuje formalnie architektury REST, lecz systematyzuje stopniowe wdrażanie jej kluczowych założeń.

Formalnych kryteriów dekompozycji systemu dostarcza metodyka projektowania dziedzinowego (ang. *Domain-Driven Design*, DDD). Jej kluczowym elementem jest *kontekst ograniczony* (ang. *Bounded Context*), rozumiany jako wyodrębniony obszar dziedziny, w którym obowiązuje jeden, spójny model pojęciowy [11]. Odwzorowanie granic kontekstu na pojedynczy mikroservis zapewnia spójność odpowiedzialności biznesowej i stanowi fundament niezależnej wdrażalności. Zasadę tę zastosowano przy podziale platformy testowej zrealizowanej w ramach niniejszej pracy.

Projektowanie w ramach kontekstu ograniczonego obejmuje również wypracowanie *wspólnego języka* (ang. *Ubiquitous Language*), czyli słownika pojęć współdzielonego przez programistów i ekspertów dziedzinowych [11]. Wspólny język ogranicza niejednoznaczności semantyczne. Przykładowo pojęcie „klient” może mieć odmienne atrybuty w module sprzedaży i w module reklamacji. Relacje i przepływ danych pomiędzy kontekstami opisuje *mapowanie kontekstów* (ang. *context mapping*). Jeżeli modele są rozbieżne, ich translację może realizować *warstwa ochronna* (ang. *anticorruption layer*), oddzielająca wewnętrzny model usługi od struktur zewnętrznych.

2.2 Porównanie z architekturą monolityczną

Architektura monolityczna (ang. *monolithic architecture*) zakłada budowę systemu jako pojedynczej, spójnej jednostki wdrożeniowej. Wszystkie moduły funkcjonalne (warstwa prezentacji, logika biznesowa oraz dostęp do danych) współdzielą jedną bazę danych i funkcjonują w ramach pojedynczego procesu systemowego [8]. Skalowanie horyzontalne takiego rozwiązania wymaga replikacji całej aplikacji, niezależnie od tego, który komponent generuje wąskie gardło wydajnościowe. Wraz z rozrostem bazy kodu wydłużeniu ulega czas kompilacji, testowania i wdrożenia, a każda wprowadzana modyfikacja niesie ryzyko destabilizacji całego systemu.

Podejście mikroservisowe eliminuje te ograniczenia poprzez dekompozycję na niezależne usługi, co zmienia zarówno strukturę kodu, jak i model operacyjny. Zestawienie kluczowych różnic pomiędzy oboma paradygmatami przedstawia tabela 2.1.

Tabela 2.1: Porównanie architektury monolitycznej i mikroserwisowej; opracowanie własne na podstawie [8, 10]

Wymiar	Architektura monolityczna	Architektura mikroserwisowa
Jednostka wdrożeniowa	Jeden artefakt obejmujący cały system	Każda usługa może być wdrażana niezależnie
Skalowanie	Replikacja całej aplikacji	Skalowanie wybranych usług proporcjonalnie do zapotrzebowania
Warstwa danych	Wspólna baza danych dla modułów systemu	Baza danych przypisana do usługi
Spójność danych	Transakcje ACID w obrębie wspólnej bazy	Transakcje lokalne oraz mechanizmy zapewniania spójności między usługami
Izolacja awarii	Awaria modułu może wpłynąć na działanie całego procesu	Awaria pojedynczej usługi może być odizolowana od pozostałych, wymagane mechanizmy odpornościowe
Stos technologiczny	Zazwyczaj jednolity dla całego systemu	Może być dobierany niezależnie dla poszczególnych usług
Komunikacja	Wywołania wewnątrzprocesowe, bez narzutu komunikacji sieciowej	Wywołania sieciowe, obciążone dodatkowym opóźnieniem i ryzykiem błędów komunikacji
Złożoność operacyjna	Niższa: pojedynczy proces i prostszy potok CI/CD	Wyższa: wiele usług, orkiestracja, obserwowalność i telemetria

Architektura mikroserwisowa zwiększa niezależność wdrożeń i możliwości skalowania kosztem większej złożoności operacyjnej. Wywołania sieciowe przenoszą część kosztu wykonania do warstwy transportowej, co ma znaczenie w komunikacji między klientem a serwerem.

2.3 Ekosystem .NET w kontekście mikroserwisów

Ewolucja platformy .NET od monolitycznego .NET Framework przez wieloplatformowy .NET Core aż po wersje zunifikowane ugruntowała jej pozycję jako środowiska do tworzenia natywnych aplikacji chmurowych (ang. *cloud-native*) [12]. Podstawę implementacyjną pracy stanowi .NET 10, edycja LTS z listopada 2025 roku [14]. Wersja ta rozwija kompilację statyczną *Native AOT* [15] oraz optymalizacje środowiska uruchomieniowego pod kątem wdrożeń kontenerowych.

Fundamentem mikroserwisów w ekosystemie .NET jest framework ASP.NET Core z serwerem Kestrel, interfejsem Minimal API oraz potokiem przetwarzania opartym na oprogramowaniu pośredniczącym (*middleware*) [12]. Wysokie pozycje Kestrela w testach TechEmpower (scenariusz *Fortunes*) [13] czynią go optymalnym wyborem dla usług o wysokich wymaganiach wydajnościowych. Ekosystem ten uzu-

pełnią biblioteka `Grpc.AspNetCore` [25] (natywne wsparcie gRPC i gRPC-Web) oraz Entity Framework Core jako warstwa mapowania obiektowo-relacyjnego.

Rozdział 3

Mechanizmy komunikacji w architekturze mikroservisowej

Jak przedstawiono w rozdziale 2, architektura mikroservisowa zakłada podział systemu na niezależne usługi odpowiedzialne za określone obszary funkcjonalne. Każdy mikroservis funkcjonuje jako odrębny proces, co sprawia, że realizacja złożonych procesów biznesowych wymaga komunikacji pomiędzy poszczególnymi usługami. Wybór odpowiedniego mechanizmu wymiany danych ma istotne znaczenie dla wydajności, skalowalności i niezawodności całego systemu [10].

W niniejszej pracy analizowana jest komunikacja synchroniczna na granicy aplikacji przeglądarkowej i usług backendowych. Jej wybór wynika z charakteru badanych operacji, w których użytkownik oczekuje na odpowiedź po wysłaniu żądania.

3.1 Komunikacja synchroniczna

Komunikacja synchroniczna opiera się na modelu żądanie–odpowiedź, w którym nadawca wysyła żądanie do odbiorcy i oczekuje na jego przetworzenie oraz zwrócenie odpowiedzi. Podejście to jest powszechnie stosowane zarówno w komunikacji pomiędzy aplikacjami internetowymi a usługami backendowymi, jak i pomiędzy mikroservisami. W ekosystemie .NET komunikacja synchroniczna może być realizowana z wykorzystaniem różnych mechanizmów, w tym stylu REST oraz mechanizmów RPC.

REST API w ASP.NET Core

Representational State Transfer (REST) to styl architektoniczny sformalizowany przez Roya Fieldinga [17], definiujący zestaw reguł dla rozproszonych systemów hipermedialnych. W architekturze mikroservisowej REST stanowi najpowszechniejszy mechanizm komunikacji synchronicznej ze względu na prostotę, uniwersalność oraz dojrzałość ekosystemu narzędziowego [8, 9]. Opiera się na wykorzystaniu

metod protokołu HTTP (GET, POST, PUT, DELETE) do operacji na zasobach identyfikowanych przez URI, których reprezentację najczęściej stanowi format JSON.

Fielding zdefiniował sześć podstawowych ograniczeń stylu REST: architekturę klient–serwer, bezstanowość, buforowalność odpowiedzi, warstwowość systemu, opcjonalny kod na żądanie oraz jednolity interfejs [17]. W praktyce inżynierskiej kluczowy element jednolitego interfejsu, jakim jest sterowanie stanem aplikacji przez hipermedia (HATEOAS), bywa pomijany na rzecz prostszego modelu żądanie–odpowiedź.

W środowisku .NET interfejsy REST API mogą być implementowane za pomocą platformy ASP.NET Core. Framework udostępnia między innymi model *Minimal API* [28], umożliwiający definiowanie punktów końcowych bez konieczności stosowania kontrolerów. Listing 3.1 przedstawia przykładową implementację asynchronicznego pobierania rekordu.

```
1 app.MapGet("/api/products/{id}", async (ProductDbContext db, string id)
  =>
2 {
3     var product = await db.Products
4         .AsNoTracking()
5         .FirstOrDefaultAsync(p => p.Id == id);
6
7     return product is null ? Results.NotFound() : Results.Ok(product);
8 });
```

Listing 3.1: Definicja punktu końcowego REST z wykorzystaniem Minimal API w .NET 10

Do głównych zalet stylu REST należą bezstanowość ułatwiająca skalowanie horyzontalne, natywna obsługa buforowania odpowiedzi w infrastrukturze sieciowej oraz czytelność formatu JSON. Do ograniczeń zalicza się narzut objętościowy danych tekstowych, brak ścisłego kontraktu binarnego oraz zjawisko nadmiernej komunikacji (ang. *chatty API*).

3.2 gRPC: architektura i implementacja

gRPC to framework komunikacji wykorzystujący protokół HTTP/2 oraz format serializacji Protocol Buffers, opracowany przez Google i udostępniony jako projekt open-source w 2015 roku [20]. W porównaniu do REST wykorzystuje binarną serializację danych oraz mechanizmy HTTP/2, w tym multipleksowanie wielu wywołań w ramach jednego połączenia TCP. Z tego względu jest często stosowany w komunikacji pomiędzy mikroserwisami, gdzie istotne są wydajność i niskie opóźnienia.

Centralnym elementem gRPC jest kontrakt definiowany za pomocą języka Pro-

tolcol Buffers, określający typy danych oraz operacje dostępne w usłudze [21]. Definicja zapisana w pliku `.proto` może zostać wykorzystana do automatycznej generacji kodu klienta i serwera. Przykładową definicję serwisu przedstawiono na listingu 3.2.

```
1 syntax = "proto3";
2
3 package product;
4
5 service ProductService {
6     rpc GetProduct (ProductRequest) returns (ProductResponse);
7     rpc StreamProducts (ProductRequest) returns (stream ProductResponse);
8 }
9
10 message ProductRequest {
11     string id = 1;
12 }
13
14 message ProductResponse {
15     string id = 1;
16     string name = 2;
17     double price = 3;
18 }
```

Listing 3.2: Definicja serwisu gRPC w formacie Protocol Buffers

W ekosystemie .NET obsługa gRPC jest dostępna za pośrednictwem pakietu `Grpc.AspNetCore`, który integruje framework z platformą ASP.NET Core. HTTP/2 zapewnia między innymi multipleksowanie wielu strumieni oraz kontrolę przepływu [26]. Kompresja nagłówków za pomocą algorytmu HPACK dodatkowo ogranicza ilość przesyłanych danych [19]. W połączeniu z binarną serializacją Protocol Buffers pozwala to zmniejszyć rozmiar komunikatów w porównaniu z formatem JSON.

Rozdział 4

gRPC-Web i komunikacja frontend-backend

Mechanizm gRPC, omówiony szczegółowo w rozdziale 3, zapewnia wysoką wydajność oraz silne typowanie w komunikacji pomiędzy usługami backendowymi. Nativny protokół gRPC opiera się jednak na mechanizmach HTTP/2, których możliwości nie są w pełni dostępne z poziomu przeglądarek internetowych. Ograniczenie to wynika ze sposobu, w jaki przeglądarki udostępniają aplikacjom internetowym dostęp do warstwy sieciowej. Interfejsy programistyczne, takie jak *Fetch API* oraz *XMLHttpRequest*, nie zapewniają bezpośredniej kontroli nad ramkami HTTP/2 ani dostępu do nagłówków końcowych (ang. *trailing headers*), wykorzystywanych przez protokół gRPC.

W konsekwencji opracowano mechanizm gRPC-Web, umożliwiający aplikacjom uruchamianym w przeglądarce komunikację z usługami wykorzystującymi gRPC. W zależności od zastosowanej implementacji komunikacja ta może być realizowana za pośrednictwem serwera pośredniczącego lub bezpośrednio przez warstwę obsługującą gRPC-Web po stronie usługi. Omówiono ograniczenia natywnego gRPC w kontekście środowiska przeglądarkowego (sekcja 4.1), architekturę rozwiązania gRPC-Web wraz z rolą serwera pośredniczącego (sekcja 4.2) oraz sposób jego wykorzystania w aplikacji frontendowej (sekcja 4.3).

4.1 Ograniczenia natywnego gRPC w przeglądarkach

Protokół gRPC w swojej natywnej postaci wykorzystuje mechanizmy transportowe HTTP/2, w tym ramki typu `HEADERS` przesyłane po zakończeniu ciała odpowiedzi. Mechanizm ten, określany jako nagłówki końcowe (ang. *trailing headers*), służy do przekazywania metadanych dotyczących statusu wywołania procedury zdalnej po przesłaniu treści odpowiedzi. Zgodnie ze specyfikacją gRPC [20], status wywo-

łania jest przekazywany za pomocą metadanych końcowych, w szczególności nagłówek `grpc-status`, natomiast opis błędu może być przekazywany za pomocą nagłówka `grpc-message`.

Problem wynika z ograniczeń środowiska przeglądarkowego. Interfejs *Fetch API*, zdefiniowany przez specyfikację WHATWG, oraz interfejs *XMLHttpRequest* nie zapewniają aplikacji bezpośredniego dostępu do warstwy transportowej HTTP/2. W szczególności kod JavaScript wykonywany w przeglądarce nie może bezpośrednio odczytywać nagłówków końcowych odpowiedzi HTTP/2. Oznacza to, że mimo możliwości zestawienia połączenia HTTP/2 przeglądarka nie udostępnia aplikacji mechanizmu pozwalającego na bezpośrednią obsługę wszystkich elementów wymaganych przez natywny protokół gRPC.

Dodatkowym ograniczeniem jest sposób zarządzania połączeniami HTTP/2 przez przeglądarkę. Przeglądarka samodzielnie zarządza połączeniami oraz multipleksowaniem strumieni, podczas gdy natywny gRPC zakłada możliwość wykorzystania mechanizmów HTTP/2 w sposób niedostępny dla standardowych interfejsów JavaScript.

Wymienione ograniczenia sprawiają, że natywny gRPC nie może być wykorzystywany bezpośrednio przez kod wykonywany w przeglądarce w taki sam sposób jak w przypadku klientów backendowych. Rozwiązaniem tego problemu jest gRPC-Web, który dostosowuje sposób transportowania komunikatów gRPC do możliwości środowiska przeglądarkowego.

Tabela 4.1 przedstawia porównanie natywnego gRPC, gRPC-Web oraz REST w kontekście komunikacji pomiędzy przeglądarką a warstwą backendową.

Tabela 4.1: Porównanie natywnego gRPC, gRPC-Web oraz REST w kontekście komunikacji frontend-backend

Cecha	Natywny gRPC	gRPC-Web	REST
Wsparcie przeglądarek	Brak	Tak	Tak (natywne)
Wersja HTTP	HTTP/2	HTTP/1.1 lub HTTP/2	HTTP/1.1 lub HTTP/2
Typ strumieniowania	Dwukierunkowe	Tylko server-side	Brak (SSE opcjonalnie)
Kontrakt i typy	Obowiązkowy, Protobuf	Obowiązkowy, Protobuf	Opcjonalny (OpenAPI), JSON
Wymagane proxy	Nie	Opcjonalne	Nie
Złożoność konfiguracji	Niska	Średnia	Niska

Jak wynika z tabeli 4.1, gRPC-Web zachowuje istotne właściwości natywnego gRPC, w szczególności wykorzystanie kontraktu opartego na *Protocol Buffers* [21]. Jednocześnie dostosowuje sposób transportowania komunikatów do ograniczeń środowiska przeglądarkowego, między innymi poprzez rezygnację z dwukierunko-

wego strumieniowania. W ekosystemie .NET możliwe jest zastosowanie zarówno serwera pośredniczącego, takiego jak Envoy, jak i bezpośredniej obsługi gRPC-Web przez usługę ASP.NET Core.

4.2 Architektura gRPC-Web

Istotną różnicą pomiędzy natywnym gRPC a gRPC-Web jest sposób przekazywania informacji o statusie wywołania. W natywnym gRPC informacje te są przekazywane w nagłówkach końcowych HTTP/2. W gRPC-Web zostały one umieszczone w specjalnej ramce znajdującej się wewnątrz ciała odpowiedzi HTTP. Zgodnie ze specyfikacją gRPC-Web ramka końcowa jest poprzedzona bajtem flagi o wartości `0x80`, natomiast ramki zawierające właściwe komunikaty oznaczane są wartością `0x00` [24].

Takie rozwiązanie pozwala na przesyłanie komunikatów gRPC z wykorzystaniem mechanizmów dostępnych dla aplikacji przeglądarkowych. Warstwa gRPC-Web może być obsługiwana bezpośrednio przez serwer aplikacyjny albo przez dedykowany serwer pośredniczący. W niniejszej pracy wykorzystano oba warianty, co umożliwia ich późniejsze porównanie pod względem wydajności.

W wariancie wykorzystującym Envoy przeglądarka wysyła żądania gRPC-Web do serwera pośredniczącego. Envoy, wyposażony w filtr `grpc_web`, odpowiada za przekształcenie komunikacji pomiędzy formatem gRPC-Web używanym przez klienta a natywnym gRPC wykorzystywanym przez usługę backendową. W wariancie Direct dodatkowa warstwa proxy nie występuje, ponieważ obsługa gRPC-Web realizowana jest bezpośrednio przez aplikację ASP.NET Core.

4.3 Implementacja we frameworku Angular

Opisane warianty gRPC-Web wymagają klienta działającego w rzeczywistym środowisku przeglądarkowym. W niniejszej pracy rolę tę pełni aplikacja frontendowa wykorzystująca framework *Angular 20*. Aplikacja odpowiada zarówno za prezentację danych, jak i obsługę komunikacji z backendem. Jej architektura została przygotowana w taki sposób, aby umożliwić przełączanie pomiędzy badanymi wariantami komunikacji bez zmiany warstwy prezentacji.

Do obsługi gRPC-Web wykorzystano bibliotekę *Connect RPC* opracowaną przez firmę Buf [31]. Biblioteka udostępnia klienta gRPC-Web dla aplikacji wykorzystujących moduły ECMAScript oraz współpracuje z generowanym kodem TypeScript. W porównaniu z klasyczną biblioteką `grpc-web` rozwiązanie to lepiej integruje się z procesem budowania zastosowanym w badanej aplikacji.

Generowanie kodu i Connect RPC

Komunikacja wykorzystująca gRPC-Web została oparta na kontrakcie zdefiniowanym w języku Protocol Buffers. Na podstawie plików `.proto` generowany jest kod TypeScript wykorzystywany następnie przez aplikację Angular. W procesie tym wykorzystano kompilator Protocol Buffers [27] oraz wtyczki `protoc-gen-es` i `protoc-gen-connect-es`. Pierwsza z nich odpowiada za generowanie definicji komunikatów, natomiast druga za generowanie deskryptorów usług wykorzystywanych przez bibliotekę Connect RPC.

Listing 4.1 przedstawia konfigurację generowania kodu klienckiego.

```
1  # Instalacja bibliotek klienckich i wtyczek generatora
2
3
4  npm install @connectrpc/connect @connectrpc/connect-web
5
6  npm install --save-dev @bufbuild/protoc-gen-es \
7    @connectrpc/protoc-gen-connect-es
8
9  # Wywołanie generatora
10
11  grpc_tools_node_protoc \
12    --plugin=protoc-gen-es=node_modules/.bin/protoc-gen-es \
13    --es_out=src/generated-connect \
14    --plugin=protoc-gen-connect-es=node_modules/.bin/protoc-gen-connect-es \
15    --connect-es_out=src/generated-connect \
16    -I ../../protos \
17    ../../protos/product.proto ../../protos/order.proto
```

Listing 4.1: Generowanie kodu klienckiego dla Connect RPC

Wywołanie generatora zostało umieszczone w skrypcie npm, dzięki czemu proces generowania kodu może być wykonywany w sposób powtarzalny podczas budowania projektu. Podejście to zapewnia również spójność pomiędzy kontraktem zapisanym w plikach Protocol Buffers a kodem wykorzystywanym przez aplikację frontendową.

Rozdział 5

Wzorce projektowe i architektoniczne

W rozdziale omówiono dwa mechanizmy zastosowane w systemie testowym i istotne dla interpretacji wyników: organizację pamięci podręcznej zgodnie ze wzorcem Cache-Aside (sekcja 5.1) oraz rolę serwera pośredniczącego Envoy Proxy (sekcja 5.2).

5.1 Wzorzec Cache-Aside

Wzorzec *Cache-Aside* polega na bezpośrednim zarządzaniu pamięcią podręczną przez warstwę aplikacji, która odpowiada za synchronizację pomiędzy magazynem podręcznym a głównym źródłem danych [29]. W trakcie operacji odczytu aplikacja weryfikuje dostępność żądanego zasobu w magazynie podręcznym na podstawie deterministycznie wygenerowanego klucza. W przypadku trafienia (ang. *cache hit*) dane są natychmiast zwracane z pamięci operacyjnej bez angażowania relacyjnej bazy danych. W sytuacji chybienia (ang. *cache miss*) aplikacja kieruje zapytanie do głównej bazy danych, po czym otrzymany wynik zapisuje w pamięci podręcznej z określonym czasem ważności (ang. *Time to Live*, TTL) i dopiero wtedy odsyła go do klienta, dzięki czemu kolejne odwołania do tego samego zasobu są już serwowane bezpośrednio z bufora.

Zastosowanie tego podejścia przynosi wymierne korzyści architektoniczne, ponieważ w pamięci podręcznej alokowane są wyłącznie dane rzeczywiście pożądane przez użytkowników, co zapobiega marnotrawieniu zasobów na rzadko odpytywane rekordy. Głównym wyzwaniem pozostaje kwestia unieważniania nieaktualnych wpisów (ang. *cache invalidation*), którą realizuje się zazwyczaj poprzez pasywne wygasanie czasowe TTL lub jawne usuwanie i aktualizowanie odpowiednich kluczy w momencie wykonywania operacji zapisu lub modyfikacji danych w głównym magazynie.

5.2 Rola serwera pośredniczącego Envoy

W rozproszonych architekturach zorientowanych na usługi serwery pośredniczące (ang. *reverse proxy* lub *API Gateway*) pełnią kluczową rolę w zarządzaniu ruchem przychodzącym, agregacji żądań oraz translacji protokołów komunikacyjnych. Narzędzia takie jak Envoy Proxy, zaimplementowane w językach natywnych o wysokiej wydajności, są powszechnie stosowane do mostkowania ruchu pomiędzy różnymi standardami sieciowymi, np. do konwersji zapytań przeglądarkowych gRPC-Web na natywny protokół gRPC oparty o HTTP/2. Zastosowanie dedykowanego serwera pośredniczącego pozwala na odseparowanie zadań infrastrukturalnych od logiki biznesowej usług backendowych, jednak wprowadza dodatkowy przeskok sieciowy (ang. *network hop*) oraz potencjalny narzut związany z przetwarzaniem i buforowaniem ramek danych. W systemie testowym Envoy realizuje jeden z dwóch porównywanych wariantów gRPC-Web.

W tej konfiguracji przeglądarka komunikuje się z Envoy przez gRPC-Web i HTTP/1.1, natomiast połączenie między serwerem pośredniczącym a usługą wykorzystuje natywny gRPC oparty na HTTP/2. Wariant Direct pomija tę warstwę i łączy klienta z usługą bezpośrednio. Oba rozwiązania uwzględniono w scenariuszach pomiarowych opisanych w rozdziale 7, ponieważ odpowiadają dwóm sposobom udostępnienia usługi gRPC aplikacji przeglądarkowej.

Obecność Envoy należy uwzględniać przy interpretacji wyników. Różnica między wariantem REST a ścieżką z serwerem pośredniczącym obejmuje jednocześnie sposób kodowania komunikatów, translację gRPC-Web oraz dodatkowy element pośredniczący. Dlatego w analizie traktowano Envoy jako część całej konfiguracji komunikacyjnej, a porównania wykonywano dla pełnych ścieżek żądania—od przeglądarki do usługi i z powrotem.

Rozdział 6

Zrealizowany system i jego wdrożenie

W rozdziale opisano system zrealizowany na potrzeby pracy: jego architekturę, kontrakty usług, topologię kontenerów, implementację pamięci podręcznej istotną dla metodyki pomiaru, model wdrożenia w środowisku chmurowym oraz warstwę obserwowalności.

6.1 Architektura badanego systemu testowego

Na potrzeby przeprowadzenia eksperymentów wydajnościowych zrealizowano autorski system testowy. Implementację podporządkowano celom badawczym, skupiając się na porównaniu wybranych protokołów i wariantów komunikacji sieciowej. Architektura systemu składa się z trzech głównych warstw: warstwy prezentacji w postaci aplikacji przeglądarkowej, warstwy usług biznesowych złożonej z mikroserwisów opartych na platformie .NET 10 oraz warstwy infrastrukturalnej obejmującej relacyjne bazy danych, pamięć podręczną i komponenty obserwowalności.

Środowisko eksperymentalne zostało rozproszone pomiędzy dwie fizyczne maszyny: warstwa mikroserwisów wraz z całą infrastrukturą pomocniczą została uruchomiona na zewnętrznym serwerze, natomiast aplikacja kliencka działała lokalnie, na stacji roboczej pełniącej rolę węzła pomiarowego.

Przyjęte podejście powinno zagwarantować, że pomiary odzwierciedlają kompletną, rzeczywistą ścieżkę przetwarzania danych, uwzględniając opóźnienia sieciowe, warstwę pośredniczącą, routing, a także procesy serializacji i deserializacji realizowane przez silnik JavaScript w przeglądarce internetowej.

Warstwa usług biznesowych

System realizuje podział na dwa mikroserwisy zgodnie z zasadą *Bounded Context* [11]:

- **ProductService** (kontekst: katalog produktów) odpowiada za operacje odczytu katalogu, implementuje wzorzec Cache-Aside oraz punkt końcowy izolujący narzut protokołu. Dane przechowywane w dedykowanej bazie produktów obejmują 2000 produktów rozłożonych na dziesięć kategorii.
- **OrderService** (kontekst: proces zamówień) odpowiada za tworzenie zamówień: waliduje żądanie, zapisuje encję zamówienia wraz z jego pozycjami. Dane przechowywane w dedykowanej bazie zamówień.

Każdy mikroserwis jest autonomiczną jednostką wdrożeniową z własnym obrazem kontenera, konfiguracją połączeń oraz migracjami bazy danych realizowanymi w podejściu Code-First.

Istotną cechą przyjętego projektu jest całkowity brak zależności pomiędzy obiema usługami: nie występują ani synchroniczne wywołania, ani wymiana komunikatów pomiędzy `ProductService` a `OrderService`. Decyzja ta wynika bezpośrednio z celu badawczego: przedmiotem pomiaru jest komunikacja na granicy klient–usługa, a nie orkiestracja procesu obejmującego wiele usług. Wprowadzenie zależności międzyusługowej wnosiłoby do pomiarów opóźnienie drugiej usługi, utrudniając izolację narzutu samego protokołu. Konsekwencją takiego podziału jest pełna niezależność wdrożeniowa obu komponentów oraz brak sprzężenia.

Podejście Contract-First

Kontrakty usług gRPC definiowane są w plikach Protocol Buffers, które stanowią wspólną definicję interfejsów systemowych. Na ich podstawie generowany jest kod serwera (C#, pakiet `Grpc.Tools`) i klienta (TypeScript, narzędzia `@bufbuild/protoc-gen-es` oraz `@connectrpc/protoc-gen-connect-es`). Podejście Contract-First umożliwia sprawdzanie zgodności typów na granicy systemu podczas kompilacji. Część niekompatybilnych zmian można dzięki temu wykryć przed uruchomieniem aplikacji.

Topologia Docker Compose

Cały system uruchamiany jest za pomocą Docker Compose, orkiestrującego siedem kontenerów. Tabela 6.1 przedstawia limity zasobów przydzielone poszczególnym kontenerom w konfiguracji wdrożeniowej.

Tabela 6.1: Limity zasobów kontenerów w konfiguracji wdrożeniowej systemu autorskiego

Kontener	Obraz	Limit CPU	Limit RAM
product-service	obraz własny (.NET 10)	4,0	3 GB
order-service	obraz własny (.NET 10)	4,0	3 GB
envoy	envoyproxy/envoy:v1.29-latest	4,0	1 GB
postgres	postgres:17-alpine	4,0	4 GB
redis	redis:7.2-alpine	1,0	512 MB
prometheus	prom/prometheus:v2.51.2	1,0	512 MB
grafana	grafana/grafana:10.4.2	1,0	512 MB

6.2 Wdrożenie i eksploatacja w środowisku chmurowym

System testowy wdrożono w publicznej chmurze obliczeniowej, w modelu infrastruktury jako usługi (ang. *infrastructure as a service*). Niniejsza sekcja przedstawia przyjęty model wdrożenia wraz z jego uzasadnieniem, przebieg wdrożenia oraz warstwę obserwowalności. Rozstrzygnięcia omówione poniżej wynikają w znacznej mierze z wymagań eksperymentu.

Uzasadnienie przyjętego modelu wdrożenia

Wszystkie komponenty systemu uruchomiono jako kontenery na pojedynczej instancji maszyny wirtualnej, zarządzane narzędziem Docker Compose w konfiguracji przedstawionej w tabeli 6.1. Rozwiązanie alternatywne, to jest wdrożenie w klastrze zarządzanym platformą Kubernetes wraz z automatycznym skalowaniem poziomym odrzucono ze względu na wymagania pomiarowe.

Mechanizmy automatycznego skalowania oraz ponownego rozmieszczania kontenerów zmieniają liczbę i lokalizację instancji w czasie działania systemu. Stała topologia zapewniała powtarzalne warunki pomiaru. Badanie dotyczyło komunikacji, dlatego nie obejmowało wpływu platformy orkiestracji.

Proces wdrożenia

System wdrożono na serwerze chmury Hetzner Cloud o ośmiu rdzeniach wirtualnych i 16 GB pamięci, z wykorzystaniem Docker Compose w konfiguracji wdrożeniowej. Proces wdrożenia obejmuje konfigurację serwera, weryfikację punktów końcowych gotowości każdego mikroservisu oraz wykonanie automatycznych migracji bazy danych przy starcie usług.

Graf zależności Docker Compose określa kolejność uruchamiania kontenerów i ogranicza liczbę przedwczesnych prób połączenia z niedostępnymi usługami.

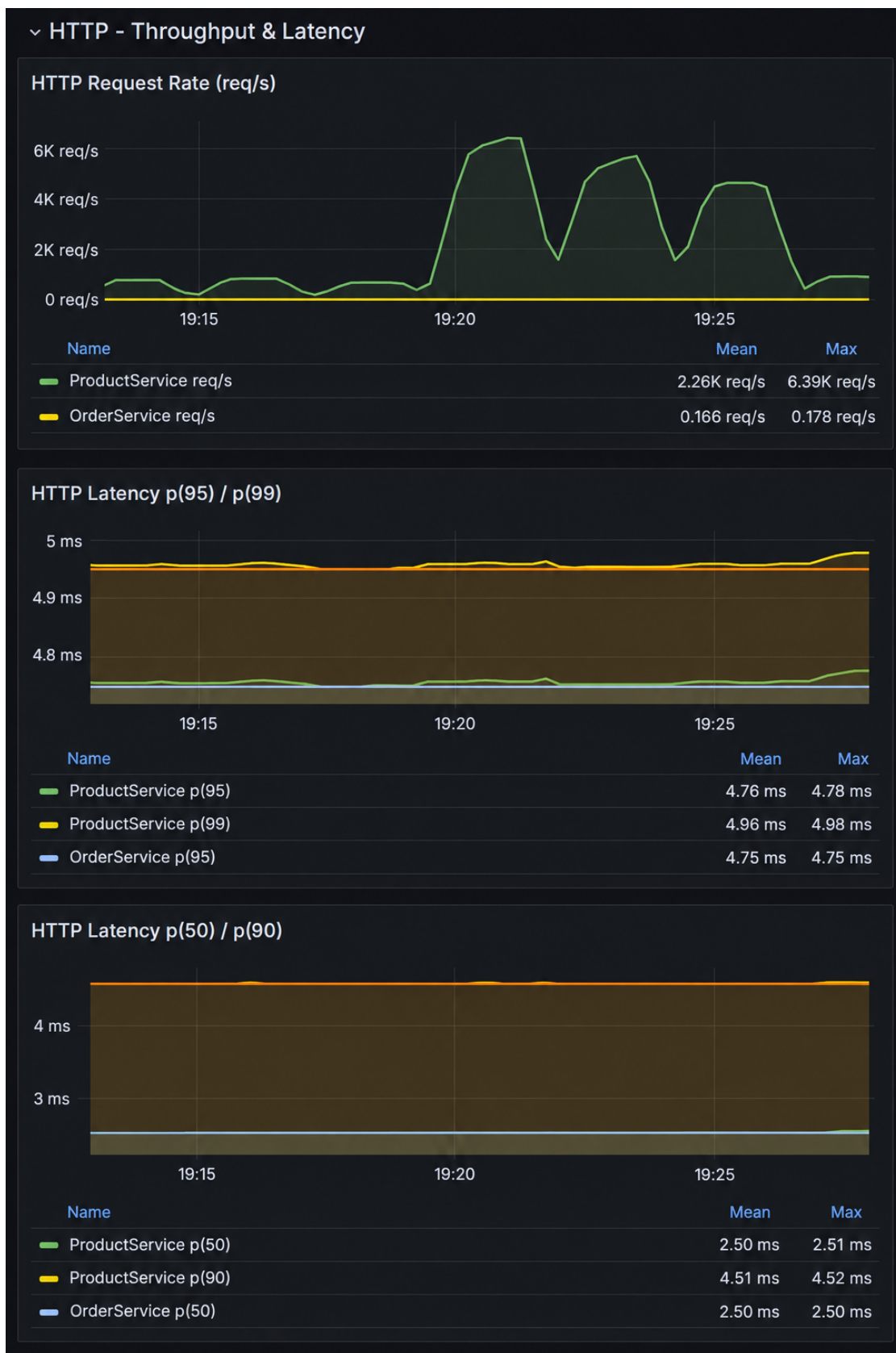
Do kontroli stanu służą punkty końcowe `/healthz/live` oraz `/healthz/ready`, udostępniane przez mechanizm wbudowany w ASP.NET Core [12].

6.3 Obserwowalność systemu

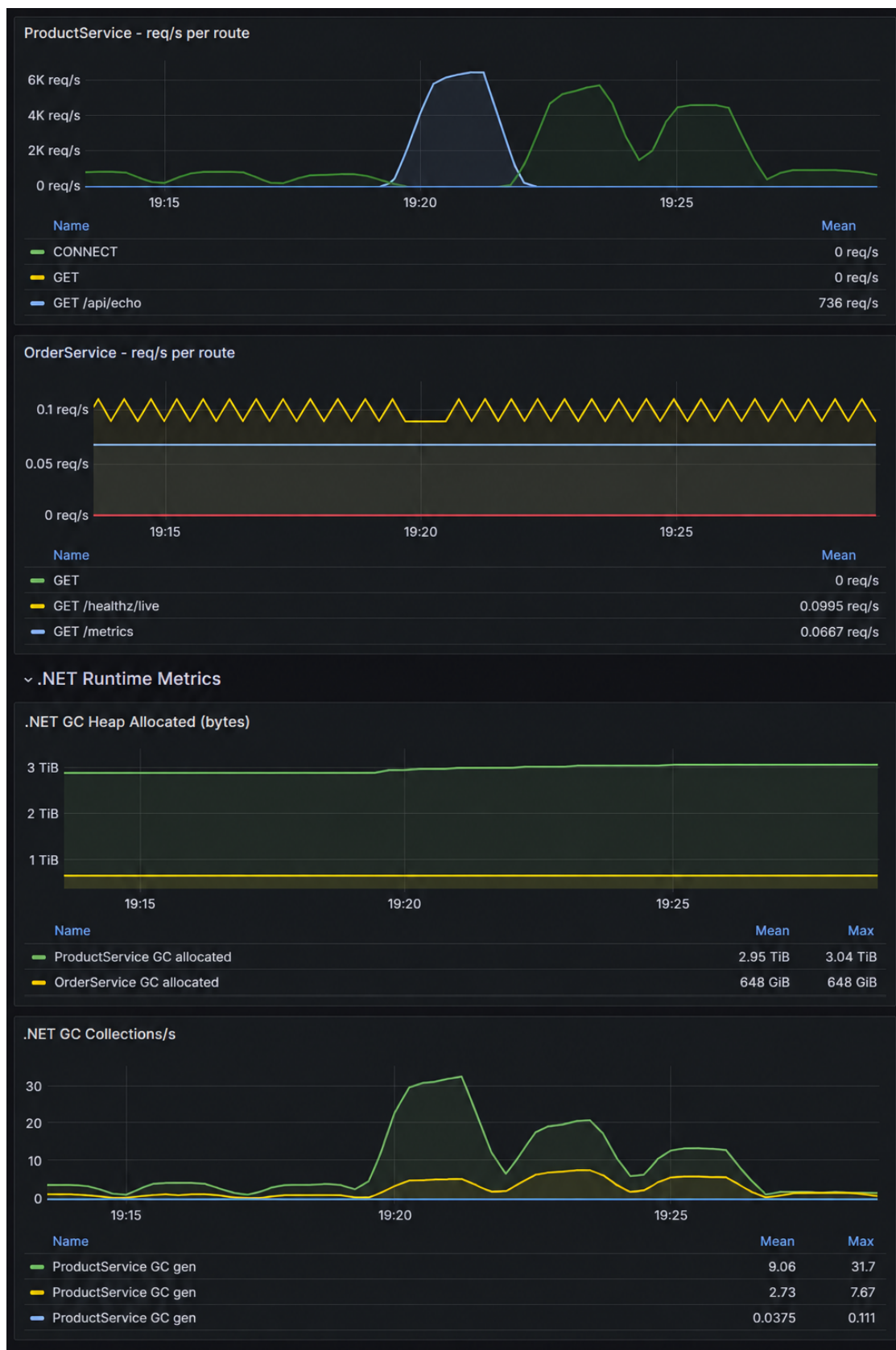
System integruje dwa filary obserwowalności zgodnie ze standardem OpenTelemetry [16], obejmując: metryki eksportowane do Prometheusa przez dedykowany punkt końcowy (czas odpowiedzi, liczba aktywnych połączeń oraz parametry środowiska uruchomieniowego .NET), a także warstwę wizualizacji danych opartą na platformie Grafana z automatycznie wdrażaną konfiguracją źródeł oraz paneli analitycznych.

Pulpity Grafany wdrażane są wraz z całym stosem, w postaci definicji przechowywanych w repozytorium i wczytywanych przy starcie kontenera. Dzięki temu warstwa wizualizacji odtwarzana jest automatycznie na każdym środowisku, bez ręcznej konfiguracji. Pierwszy z pulpitów (rysunek 6.1) obejmuje charakterystykę warstwy HTTP obu mikroserwisów: liczbę żądań na sekundę oraz percentyle czasu obsługi żądania mierzone po stronie serwera. Drugi (rysunek 6.2) przedstawia rozbięcie liczby żądań na poszczególne trasy oraz metryki środowiska uruchomieniowego .NET: wolumen alokacji pamięci i częstość usuwania zbędnych obiektów z pamięci aplikacji.

Przebiegi zarejestrowano podczas obciążeniowej weryfikacji stosu obserwowalności, poza właściwą macierzą badawczą. Ilustrują działanie warstwy monitorowania i nie są wynikami eksperymentu. Warstwa obserwowalności pełniła funkcję pomocniczą.



Rysunek 6.1: Pulpit Grafany: liczba żądań na sekundę oraz percentyle czasu obsługi żądania po stronie usług ProductService i OrderService



Rysunek 6.2: Pulpit Grafany: liczba żądań w podziale na trasy oraz metryki środowiska uruchomieniowego .NET, wolumen alokacji i częstość odśmiecania pamięci

Rozdział 7

Metodologia badań i projekt eksperymentu

Metodologia przeprowadzonego eksperymentu obejmuje cele i hipotezy badawcze, projekt aplikacji testowej, metodykę pomiaru, macierz scenariuszy, środowisko testowe, przyjęte metryki, sposób agregacji wyników oraz ograniczenia metodologiczne.

7.1 Cele i hipotezy badawcze

Głównym celem badawczym pracy jest porównawcza analiza wydajności protokołów komunikacji dostępnych dla aplikacji przeglądarkowej współpracującej z mikroserwisami zrealizowanymi w ekosystemie .NET 10. Przedmiotem pomiaru są trzy warianty:

1. REST API: HTTP/1.1 z serializacją tekstową JSON,
2. gRPC-Web przez proxy Envoy: HTTP/1.1 po stronie przeglądarki, translacja na HTTP/2 w warstwie pośredniczącej i serializacja Protocol Buffers,
3. gRPC-Web Direct: bezpośrednia obsługa w oprogramowaniu pośredniczącym ASP.NET Core, serializacja Protocol Buffers i protokół HTTP/2.

Na podstawie przeglądu literatury oraz analizy specyfikacji protokołów sformułowano następujące hipotezy badawcze:

H1: Warianty gRPC-Web osiągają niższe opóźnienie niż REST w komunikacji typu żądanie–odpowiedź, a przewaga ta rośnie wraz z rozmiarem odpowiedzi, ponieważ binarna serializacja Protocol Buffers redukuje wolumen przesyłanych danych względem formatu JSON [21].

H2: Przy odpowiedziach o niewielkim rozmiarze różnice pomiędzy protokołami są praktycznie nieistotne, ponieważ w komunikacji przez łącze rozległe czas od-

powiedzi zdominowany jest przez opóźnienie propagacji, wspólne dla wszystkich protokołów.

H3: Wariant gRPC-Web przez Envoy uzyskuje przewagę względem REST także wtedy, gdy na odcinku pomiędzy przeglądarką a proxy wykorzystywany jest HTTP/1.1. Weryfikacja hipotezy opiera się na porównaniu trzech wdrożonych konfiguracji. Nie służy ona wyznaczeniu odrębnego, przyczynowego udziału serializacji, wersji HTTP, TLS i obecności proxy.

H4: W operacjach modyfikacji stanu (ścieżka zapisu) różnice wydajnościowe pomiędzy protokołami stają się pomijalne, ponieważ całkowity czas odpowiedzi jest zdominowany przez czas wykonania transakcji bazodanowej.

H5: Wariant gRPC-Web Direct osiąga niższe opóźnienie niż wariant z proxy Envoy, ponieważ eliminuje dodatkowy skok sieciowy oraz koszt translacji realizowanej w odrębnym procesie.

7.2 Adaptacja systemu na potrzeby eksperymentu

Platforma testowa, której pełną architekturę i wdrożenie opisano w poprzednim rozdziale, wymagała wprowadzenia dedykowanych rozwiązań inżynierskich na potrzeby procedury badawczej. Modyfikacje te podporządkowano zasadzie kontroli zmiennych oraz eliminacji czynników zakłócających, co pozwoliło na wyizolowanie wielu aspektów narzutu poszczególnych protokołów komunikacyjnych.

Izolacja zmiennych i symetria przetwarzania

Ocena narzutu komunikacji wymagała kontroli różnic w przetwarzaniu danych po stronie serwera. Scenariusz Echo pomija warstwę danych, a scenariusze Products uwzględniają koszty dostępu do bazy i obsługi pamięci podręcznej właściwe dla każdej implementacji.

Punkt odniesienia o zerowym narzucie wejścia-wyjścia (scenariusz Echo)

W celu zbadania czystego kosztu serializacji, deserializacji oraz nagłówek protokołów zaimplementowano punkt końcowy `/api/echo` oraz odpowiadającą mu procedurę gRPC `EchoProducts`. Zwracają one kolekcję obiektów generowaną bezpośrednio w pamięci operacyjnej, pomijając zapytania do bazy PostgreSQL oraz pamięci podręcznej Redis. Listing 7.1 przedstawia implementację tego punktu końcowego.

```
1 app.MapGet("/api/echo", (int count = 200) =>
2 {
3     var products = Enumerable.Range(1, count).Select(i => new
4     {
```

```

5         id = i.ToString(),
6         name = $"Echo_Product_{i}",
7         description = $"This_is_a_hardcoded_echo_product_number_{i}_for_
           benchmarking_protocol_overhead",
8         price = 9.99 * i,
9         categoryId = "echo",
10        stock = 100
11    });
12
13    return Results.Ok(new { products, totalCount = count });
14 });

```

Listing 7.1: Punkt końcowy Echo izolujący narzut protokołu od warstwy danych

Reprezentacja danych w pamięci podręcznej Ścieżki REST i gRPC przechowują w Redis odrębne reprezentacje odpowiedzi. Endpoint REST zapisuje gotowy dokument JSON pod kluczem z prefiksem `rest_products_`. Przy trafieniu odczytuje tekst przez `GetStringAsync` i zwraca go przez `Results.Content`, bez deserializacji JSON do obiektów i ponownej serializacji JSON. Usługa gRPC zapisuje bajty Protocol Buffers pod kluczem z prefiksem `grpcpb_products_`. Przy trafieniu odtwarza obiekt przez `ListProductsResponse.Parser.ParseFrom`, po czym warstwa gRPC serializuje go do odpowiedzi. Warianty Envoy i Direct korzystają z tej samej implementacji usługi. Obie reprezentacje mają pięciominutowy czas ważności, ale koszt przetwarzania trafienia pozostaje różny. Zapis binarny w ścieżce gRPC usuwa dodatkową konwersję z JSON, nie zrównuje jednak jej kosztu z obsługą REST.

Dobór wolumenu danych testowych Baza danych katalogu produktów została zainicjalizowana zbiorem 2000 rekordów. Wartość ta wyznacza górną granicę rozmiaru strony wyników, co umożliwia generowanie odpowiedzi o rozmiarze przekraczającym 260 kB. Pozwala to na weryfikację zachowania protokołów przy przekroczeniu początkowego okna przeciążenia TCP (ang. *Initial Congestion Window*).

Aplikacja kliencka jako narzędzie pomiarowe

Podstawowym założeniem metodologicznym pracy jest wykorzystanie rzeczywistego środowiska przeglądarki internetowej w roli klienta pomiarowego. Zastosowanie dedykowanych bibliotek generatorów obciążenia odtwarzałoby ruch sieciowy, lecz pomijało kluczowe elementy determinujące wydajność aplikacji webowej: narzut silnika JavaScript, koszt deserializacji w pamięci przeglądarki oraz cykl renderowania drzewa DOM.

Do realizacji pomiarów wykorzystano aplikację kliencką we frameworku Angular 20, działającą w architekturze bezstrefowej (ang. *zoneless*). Wyeliminowanie biblioteki Zone.js pozwoliło usunąć wewnętrzny narzut modyfikacji zdarzeń asynchronicznych, który mógłby zafałszować pomiary czasu wykonania żądań. Komunikację zrealizowano przy użyciu biblioteki Connect RPC [31].

Wykonanie scenariuszy testowych pod zadaniem obciążeniem zautomatyzowano przy użyciu biblioteki Playwright [30], kontrolującej przeglądarkę Chromium w trybie bezokienkowym. Każdy równoległy wątek testowy uruchamiano w izolowanym kontekście przeglądarki z odrębnym magazynem sesji i pamięcią podręczną, co zapobiegało interferencji pomiędzy równoległymi pomiarami.

7.3 Metodyka pomiaru

Czas wykonania żądań mierzono bezpośrednio w kontekście wykonawczym strony przy użyciu interfejsu `performance.now()`. Jest to zegar przeznaczony do pomiarów wydajnościowych, którego rozdzielczość jest wystarczająca do rejestrowania czasów odpowiedzi uzyskiwanych w przeprowadzonych testach.

Początek pomiaru wyznacza moment rozpoczęcia obsługi żądania przez aplikację, natomiast jego koniec odpowiada chwili, w której dane otrzymane z serwera zostają naniesione na drzewo dokumentu (DOM). Do wykrycia tego momentu wykorzystano mechanizm stosowany w aplikacji Angular. Po zakończeniu aktualizacji widoku komponent inkrementuje atrybut `data-load-id`, którego zmianę obserwuje instancja `MutationObserver` działająca w tym samym wątku interfejsu.

Pomiar wykonany bezpośrednio w stronie pozwala uniknąć konieczności zewnętrznego odpytywania stanu drzewa DOM za pośrednictwem protokołu Chrome DevTools (CDP) [32]. W przypadku takiego rozwiązania cykliczne odpytywanie (ang. *polling*) wiązałoby się z dodatkową komunikacją pomiędzy narzędziem sterującym a przeglądarką.

Takie podejście pozwoliło ograniczyć wpływ zewnętrznych narzędzi na wyniki pomiarów oraz zastosować ten sam sposób wyznaczania końca pomiaru dla wszystkich badanych wariantów komunikacji. Rejestrowany czas obejmuje obsługę żądania po stronie aplikacji, w tym odbiór odpowiedzi, jej deserializację oraz aktualizację widoku.

Sterowanie scenariuszem

Parametry scenariusza (rozmiar strony wyników, stan pamięci podręcznej, liczba pozycji zamówienia oraz rozmiar odpowiedzi punktu końcowego Echo) przekazywane są aplikacji przez parametry adresu URL. Rozwiązanie to zapewnia, że każda

konfiguracja macierzy badawczej jest osiągalna zarówno automatycznie, jak i ręcznie, przez otwarcie odpowiedniego adresu w przeglądarce, co ułatwia weryfikację poprawności pomiaru.

Kontrola czynników zakłócających

W projekcie eksperymentu zastosowano sześć mechanizmów ograniczających wpływ czynników niebędących przedmiotem badania.

Rozgrzewka. Przed rozpoczęciem pomiarów wykonywanych jest dwadzieścia żądań na każdy protokół w każdym scenariuszu. Celem jest doprowadzenie kompilacji just-in-time platformy .NET oraz pul połączeń do stanu ustalonego.

Zapytania przygotowawcze. W każdej komórce macierzy każdy z równoległych klientów wykonuje trzy nierejestrowane żądania. Wypełniają one pamięć podręczną dla danego rozmiaru strony, dzięki czemu scenariusz z ciepłą pamięcią podręczną nie rozpoczyna się od chybienia.

Rotacja kolejności protokołów. Kolejność badania protokołów zmienia się między przebiegami zgodnie ze schematem kwadratu łacińskiego. Ogranicza to ryzyko systematycznego uprzywilejowania protokołu mierzonego jako ostatni, gdy system osiągnął już stan ustalony. W literaturze zjawisko to określa się jako efekt kolejności [34].

Przerwy między przebiegami. Kolejne przebiegi całej macierzy rozdziela piętnastosekundowa przerwa przeznaczona na ustabilizowanie mechanizmu odświeżania pamięci i pul połączeń.

Adaptacyjny czas namysłu. Pomędzy kolejnymi żdaniami tego samego klienta zachowywana jest przerwa odwzorowująca czas namysłu użytkownika. Jej wartość jest wyliczana dla każdej komórki macierzy tak, aby zapotrzebowanie na pasmo nie przekroczyło połowy zmierzonej przepustowości łącza.

Wykluczenie obsługi z pamięci podręcznej przeglądarki. Powtarzane żądania o identycznym adresie mogłyby teoretycznie zostać obsłużone lokalnie, bez udziału sieci, co unieważniłoby pomiar. Odpowiedzi nie zawierają nagłówków ustanawiających okres świeżości, przeglądarka nie ma zatem podstaw do ponownego użycia zapisanej treści, a każdy kontekst dysponuje ponadto własną, odrębną pamięcią podręczną.

Budżet pasma łącza klienta

Zapotrzebowanie pojedynczej komórki macierzy na pasmo jest iloczynem liczby równoległych klientów i rozmiaru odpowiedzi, podzielonym przez długość cyklu żądania.

Nasycenie łącza klienta wprowadzałoby błąd systematyczny. REST przesyła

większe odpowiedzi, więc ograniczona przepustowość zawyżałaby zmierzoną przepagę gRPC-Web. Częstość żądań dobrano tak, aby uniknąć tego efektu.

Miarą podstawową jest opóźnienie pojedynczego żądania. Aby zachować wszystkie konfiguracje bez nasycania łącza, zmniejszono częstość żądań za pomocą czasu namysłu wyliczanego z zależności

$$t = \max \left(100 \text{ ms}, \frac{n \cdot s}{0,5 \cdot B} \right), \quad (7.1)$$

gdzie t oznacza czas namysłu, n liczbę równoległych klientów, s rozmiar odpowiedzi w bajtach, a B zmierzoną przepustowość łącza wyrażoną w bajtach na sekundę. Pominięcie opóźnienia w mianowniku daje oszacowanie zachowawcze: rzeczywisty cykl trwa dłużej, więc faktyczne obciążenie łącza jest niższe od przyjętego budżetu.

Dla operacji zapisu przyjęto odrębny, niższy budżet, ponieważ scenariusz ten obciąża kierunek wysyłania, słabszy na łączach asymetrycznych.

7.4 Scenariusze i macierz badawcza

Eksperyment obejmuje trzy kategorie scenariuszy, z których każda bada inny aspekt wydajności systemu.

Scenariusz Echo: narzut komunikacji

Scenariusze Echo stanowią fundament eksperymentu, ponieważ izolują koszt protokołu od kosztu warstwy danych. Odpowiedź generowana jest w pamięci, bez udziału bazy danych i pamięci podręcznej. Scenariusz ten świadomie nie renderuje pozycji w postaci tabeli.

Scenariusz Products: odczyt katalogu

Scenariusze Products obejmują odczyt katalogu produktów z Redis albo, przy wymuszonym pominięciu pamięci podręcznej, z PostgreSQL. Przy trafieniu REST wykorzystuje gotowy JSON, a gRPC odtwarza wiadomość binarną i ponownie ją serializuje. Przy odczycie z bazy obie ścieżki budują odpowiedź i serializują ją do własnego formatu. Dalszy pomiar obejmuje transmisję, deserializację w przeglądarce i aktualizację interfejsu.

Stan pamięci podręcznej sterowany jest nagłówkiem `X-Bypass-Cache`, którego obecność wymusza zapytanie do bazy danych. Rozwiązanie to pozwala badać oba stany bez restartu systemu.

Scenariusz Orders: zapis zamówienia

Scenariusze Orders badają operacje zapisu angażujące pełen łańcuch transakcyjny: walidację i zapis encji zamówienia wraz z pozycjami. Pozycje zamówienia są losowane z ustalonej puli produktów inicjalizacyjnych, co pozwala zmieniać rozmiar żądania w powtarzalny sposób.

Wymiary macierzy

Tabela 7.1 przedstawia wymiary macierzy badawczej.

Tabela 7.1: Wymiary macierzy badawczej

Wymiar	Wartości
Protokoły	REST, gRPC-Web/Envoy, gRPC-Web/Direct
Liczba równoległych klientów	10, 20
Rozmiar odpowiedzi Echo	10, 100, 200, 500, 2000 rekordów
Rozmiar strony Products	10, 100, 200, 500, 1000, 2000 produktów
Stan pamięci podręcznej	ciepła (Redis), zimna (PostgreSQL)
Pozycje zamówienia	1, 10, 50, 200
Liczba powtórzeń macierzy	5
Żądań na klienta w komórce	20

Macierz obejmuje 39 konfiguracji, co przy trzech protokołach i pięciu powtórzeniach daje 585 wykonanych testów oraz 171 tysięcy zarejestrowanych pomiarów. Łączny czas przebiegu wyniósł 69,7 minuty. Odrzucono pięć konfiguracji, w których iloczyn obciążenia i rozmiaru odpowiedzi przekraczał przyjęty próg.

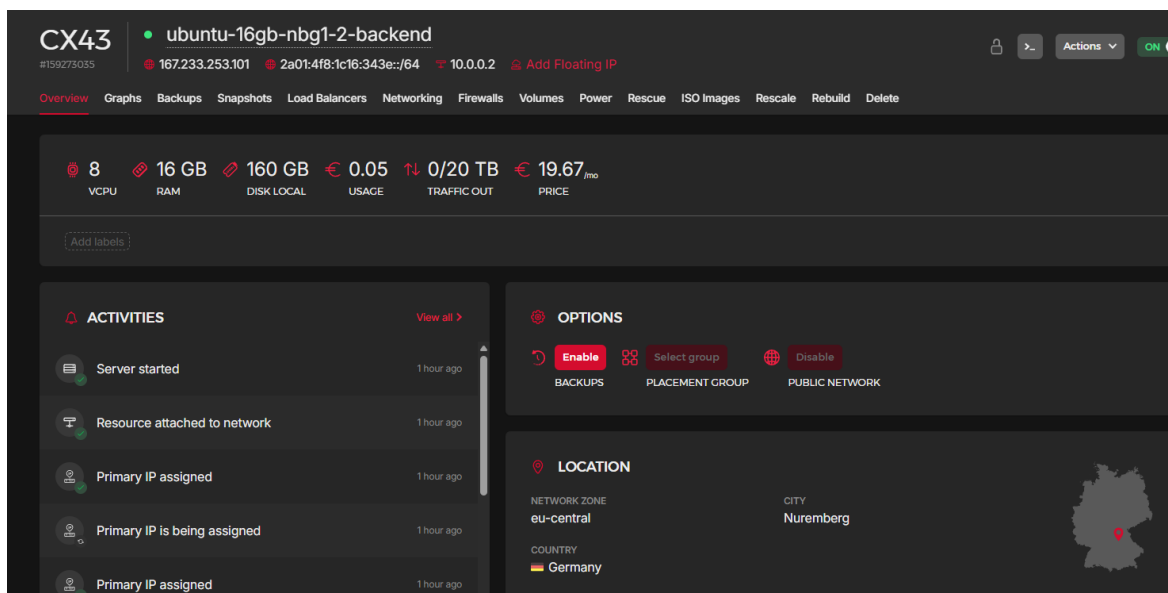
Liczba powtórzeń macierzy równa pięciu pozwala wyznaczyć odchylenie standardowe median przebiegów, a zatem ocenić powtarzalność pomiaru, nie zaś wyłącznie jego wartość centralną.

Liczbę żądań na klienta ustalono na dwadzieścia, co przy badanych poziomach obciążenia daje od 200 do 400 próbek na komórkę w jednym przebiegu, a zatem od 1000 do 2000 próbek na konfigurację i protokół po uwzględnieniu wszystkich powtórzeń.

7.5 Środowisko testowe

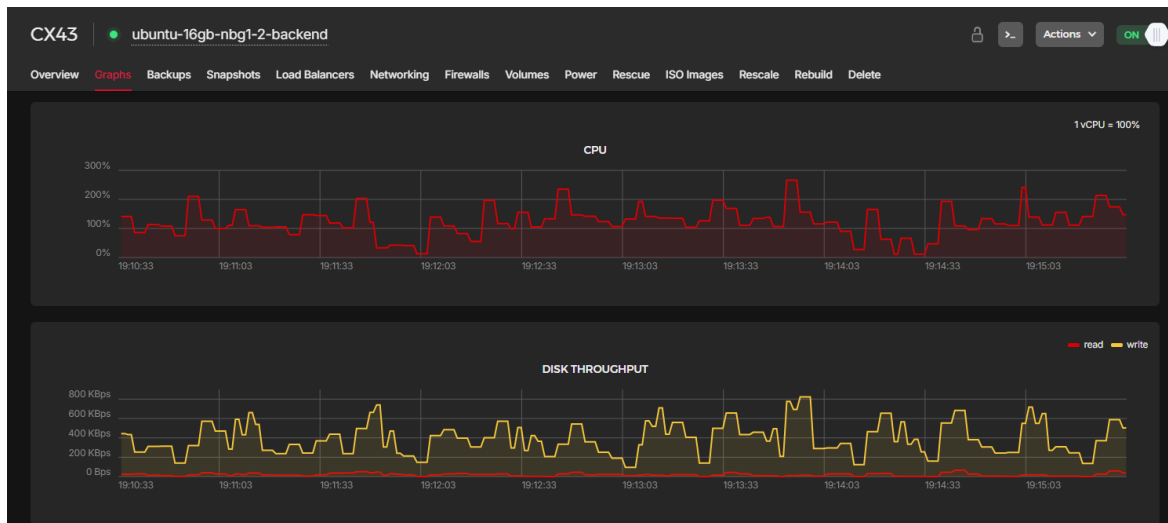
Konfiguracja sprzętowa i sieciowa

System testowany wdrożono na serwerze chmury Hetzner Cloud o ośmiu rdzeniach wirtualnych i 16 GB pamięci, zlokalizowanym w Norymberdze, pracującym pod kontrolą systemu Ubuntu 24.04.



Rysunek 7.1: Konsola Hetzner Cloud: parametry serwera, na którym wdrożono system testowany

Parametry maszyny w postaci udostępnianej przez konsolę dostawcy przedstawia rysunek 7.1. Jest to instancja typu CX43 o ośmiu rdzeniach wirtualnych, 16 GB pamięci i dysku lokalnym o pojemności 160 GB, umieszczona w strefie eu-central.



Rysunek 7.2: Wykorzystanie procesora oraz przepustowość dysku serwera w trakcie pracy systemu pod obciążeniem (pulpit konsoli Hetzner Cloud)

Klientem pomiarowym jest stacja robocza autora, łącząca się z serwerem przez publiczną sieć rozległą. Konfiguracja odwzorowuje dostęp użytkownika do zdalnej usługi i zachowuje wpływ opóźnienia sieciowego. Umieszczenie obu stanowisk w jednym centrum danych zmniejszyłoby RTT, a tym samym zakres obserwowanego efektu związanego z liczbą obiegów pakietu.

Stanowisko klienckie stanowi komputer o procesorze AMD Ryzen 7 5700X

(8 rdzeni, 16 wątków logicznych, częstotliwość bazowa 3,4 GHz) oraz 32 GB pamięci operacyjnej, pracujący pod kontrolą systemu Microsoft Windows 11 (kompilacja 26200).

Parametry stanowiska klienckiego mają znaczenie dla interpretacji wyników, ponieważ, jak wykazano w sekcji 8.7, przy wyższych poziomach współbieżności czynnikiem ograniczającym przepustowość pomiaru było stanowisko klienckie, nie system testowany. Liczba rdzeni fizycznych stanowi przy tym prawdopodobny mechanizm tego zjawiska: każdy równoległy użytkownik odwzorowany jest odrębnym kontekstem przeglądarki z własnym wątkiem renderującym, zatem już przy dwudziestu kontekstach liczba wątków wykonujących pracę obliczeniową przewyższa liczbę dostępnych rdzeni. Samego poziomu granicznego nie określono jednak na podstawie liczby rdzeni. Został on wyznaczony eksperymentalnie poprzez obserwację zachowania stanowiska klienckiego przy rosnącym poziomie współbieżności, zgodnie z metodą opisaną w przywołanej sekcji.

Zmierzona charakterystyka łącza

Parametry łącza wyznaczono pomiarowo, ponieważ stanowią one wielkość wejściową dla mechanizmu opisanego w sekcji 7.3 oraz punkt odniesienia dla interpretacji wyników. Pomiar powtórzono trzykrotnie, rozrzut wyników nie przekroczył pięciu procent.

Tabela 7.2: Zmierzona charakterystyka łącza klient–serwer

Wielkość	Wartość
Opóźnienie obiegu pakietu (RTT)	26,2–26,9 ms (zakres 24–30 ms)
Pobieranie, jeden strumień	19,6–20,8 Mbit/s
Pobieranie, cztery strumienie	74–79 Mbit/s
Pobieranie, osiem strumieni	143–146 Mbit/s
Pobieranie, szesnaście strumieni	269–273 Mbit/s
Wysyłanie, jeden strumień	19,2 Mbit/s

Przepustowość pojedynczego połączenia wynosi około 20 Mbit/s. Wartość zgregowana rośnie wraz z liczbą strumieni i dla szesnastu przekracza 270 Mbit/s. Wynik pojedynczego strumienia odpowiada iloczynowi opóźnienia i przepustowości omówionemu w sekcji 8.2. Podobny wynik uzyskano dla wysyłania danych.

Wartość z pojedynczego strumienia jest dolnym ograniczeniem pojemności łącza. W pobieraniu wynosi ona 20 Mbit/s wobec 273 Mbit/s uzyskanych dla wielu strumieni.

Zweryfikowana warstwa transportowa

Wersję protokołu HTTP negocjowaną przez przeglądarkę na każdej ze ścieżek zweryfikowano pomiarowo, korzystając z protokołu Chrome DevTools [32]. Wyniki przedstawia tabela 7.3.

Tabela 7.3: Zweryfikowana warstwa transportowa poszczególnych ścieżek komunikacji

Ścieżka	Port	Wersja HTTP	Połączeń TCP
REST	:5000	http/1.1	1
gRPC-Web przez Envoy	:8080	http/1.1	1
gRPC-Web Direct	:5002	h2	1

W przeprowadzonych testach ścieżka bezpośrednia korzystała z protokołu HTTP/2 negocjowanego za pomocą mechanizmu ALPN w ramach połączenia TLS, natomiast pozostałe dwie ścieżki wykorzystywały HTTP/1.1. Różnica ta wynika z właściwości badanych rozwiązań, ponieważ natywne gRPC korzysta z HTTP/2. Wariant gRPC-Web z proxy Envoy pozwala natomiast przysyłać ładunek binarny przy wykorzystaniu po stronie przeglądarki protokołu HTTP/1.1. Umożliwia to porównanie wariantów różniących się zarówno formatem serializacji, jak i warstwą transportową.

7.6 Metryki i sposób agregacji wyników

Dla każdej komórki macierzy i każdego protokołu wyznaczane są: liczba pomiarów, wartość minimalna, średnia arytmetyczna, mediana, percentyle 90., 95. i 99., wartość maksymalna, przepustowość wyrażona liczbą zakończonych operacji na sekundę, liczba żądań zakończonych niepowodzeniem.

Miarą podstawową w analizie przedstawionej w rozdziale 8 jest mediana, nie średnia arytmetyczna. Wybór ten wynika z odporności mediany na pojedyncze wartości odstające, które mogą pojawić się w pomiarach czasu odpowiedzi. Dzięki temu pojedyncze nietypowo wysokie wartości mają mniejszy wpływ na wartość reprezentującą typowy czas wykonania operacji.

Percentyl 95. nie został przyjęty jako główna miara porównawcza, ponieważ jest silniej zależny od zachowania górnego ogona rozkładu. Wartości percentyli 90., 95. i 99. są jednak zapisywane dla każdej komórki macierzy i mogą być wykorzystane w dodatkowej analizie rozkładu czasów odpowiedzi.

Powtarzalność pomiaru wyrażono współczynnikiem zmienności median pięciu przebiegów, zdefiniowanym jako iloraz odchylenia standardowego i średniej. Ze względu na niewielką liczbę powtórzeń zastosowano odchylenie standardowe z próby, z dzielnikiem $n - 1$, stanowiące estymator odchylenia populacyjnego.

Czas namysłu jest wyliczany osobno dla każdej komórki macierzy, dlatego wartości przepustowości można porównywać między wariantami komunikacji tylko w obrębie tej samej komórki. użytą wartość zapisano w wynikach. Ograniczenie nie dotyczy pomiarów opóźnienia.

Wyniki zapisywane są w formacie JSON, obejmującym zarówno statystyki zbiorcze ze wszystkich przebiegów, jak i statystyki każdego przebiegu osobno, co umożliwia weryfikację, czy dana relacja pomiędzy protokołami zachodzi konsekwentnie, a nie wyłącznie w zestawieniu uśrednionym.

Przedziały ufności i kryterium rozstrzygalności

Niepewność wartości centralnej oceniono za pomocą przedziałów ufności. Jednostką analizy jest mediana pojedynczego przebiegu.

Wybór tej jednostki jest istotny metodycznie. Pomiary wykonane w obrębie jednego przebiegu nie są wzajemnie niezależne: dzielą stan pamięci podręcznej, ten sam kontekst przeglądarki oraz te same chwilowe warunki sieciowe. Wyznaczenie przedziału z kilkuset próbek jednej komórki dałoby wynik pozornie precyzyjny, lecz nieuprawniony, ponieważ efektywna liczba niezależnych obserwacji jest znacznie mniejsza od liczby próbek. Niezależnych replikacji jest w eksperymencie pięć: tyle, ile przebiegów całej macierzy, każdy z odrębną kolejnością protokołów. Zgodnie z zasadami projektowania eksperymentów [34] to one stanowią jednostkę analizy.

Dla pięciu median przebiegów wyznaczono średnią $\bar{m}$ oraz odchylenie standardowe z próby s , a następnie dwustronny przedział ufności na poziomie 95%:

$$\bar{m} \pm t \cdot \frac{s}{\sqrt{k}}, \quad k = 5, \quad t = 2,776, \quad (7.2)$$

gdzie t oznacza wartość krytyczną rozkładu Studenta dla czterech stopni swobody. Przy pięciu obserwacjach mnożnik ten jest znacznie większy od wartości granicznej 1,96, co czyni oszacowanie zachowawczym.

Porównania pomiędzy protokołami przeprowadzono w układzie sparowanym. Ponieważ w każdym przebiegu mierzone są wszystkie trzy protokoły w tych samych warunkach, dla każdej pary wyznaczono pięć różnic w obrębie przebiegu. Parowanie eliminuje zmienność pomiędzy przebiegami, wspólną dla wszystkich protokołów, dzięki czemu przedział dla różnicy jest istotnie węższy niż przedział wyznaczony z dwóch niezależnych szeregów.

Różnicę uznaje się za rozstrzygniętą, jeżeli jej przedział ufności nie zawiera zera. Kryterium wiąże ocenę bezpośrednio ze zmiennością danych.

Przy pięciu obserwacjach najmniejsza osiągalna dwustronna wartość prawdopodobieństwa w teście rangowym dla par wynosi 0,0625. Test nie mógłby osiągnąć przyjętego poziomu istotności, dlatego raportowane są przedziały ufności.

7.7 Ograniczenia metodologiczne

Przyjęta metodyka pozwala porównać badane protokoły w kontrolowanych i powtarzalnych warunkach, lecz wyznacza również granice uogólniania wyników. Ograniczenia te nie podważają porównań wykonanych w ramach eksperymentu, ale określają sytuacje, w których przeniesienie wniosków na inne systemy wymaga dodatkowej walidacji.

Zakres uogólniania wyników ogranicza fakt, że badanie przeprowadzono dla jednej implementacji systemu, jednego stosu technologicznego (.NET 10, Angular 20 i Chromium) oraz jednej konfiguracji sprzętowej. Dzięki identycznej logice biznesowej i wspólnej aplikacji klienckiej ograniczono liczbę zmiennych zakłócających, jednak wyniki nie opisują wszystkich implementacji REST i gRPC-Web ani zachowania innych silników przeglądarek. W szczególności koszt serializacji, deserializacji i aktualizacji widoku może zależeć od bibliotek klienckich, modelu danych i środowiska wykonawczego. W ciepłym odczycie Products występuje dodatkowo asymetria obsługi Redis: REST zwraca gotowy JSON, a gRPC odtwarza obiekt z bajtów i ponownie go serializuje. Wynik tego scenariusza obejmuje koszt obu implementacji pamięci podręcznej, więc nie wyznacza samodzielnie wpływu formatu transmisji. Scenariusz Echo pomija Redis i stanowi odrębny punkt odniesienia.

Warunki sieciowe stanowią odrębne ograniczenie, ponieważ pomiary wykonano dla jednej rzeczywistej ścieżki pomiędzy stanowiskiem klienckim a serwerem. Jej opóźnienie obiegu pakietu wynosiło 26,2–26,9 ms, a przepustowość pojedynczego strumienia około 20 Mbit/s. Wykazany mechanizm zależy od liczby okien odbiorczych i kosztu kolejnych obiegów pakietu, dlatego wartości bezwzględnych oraz procentowej przewagi nie należy bezpośrednio przenosić na sieci lokalne, połączenia mobilne ani łącza o innym iloczynnie opóźnienia i przepustowości.

Na zakres interpretacji wpływa również brak kompresji ciała odpowiedzi na badanych ścieżkach. Zapewniło to symetrię porównania, ale ogranicza zakres wniosków o opóźnieniu do transferów nieskompresowanych. Analiza objętościowa w sekcji 8.1 wskazuje, że po zastosowaniu gzip wszystkie badane ładunki mieściłyby się w jednym oknie odbiorczym, lecz wpływ kompresji na czas odpowiedzi nie został zmierzony eksperymentalnie. Nie uwzględniono więc także kosztu procesora związanego z kompresją i dekompresją.

Zakres analizowanego obciążenia obejmuje dziesięciu i dwudziestu równoległych klientów. Wyższe poziomy współbieżności wyłączono po ustaleniu, że powyżej dwudziestu klientów przepustowość ogranicza stanowisko pomiarowe, a nie system testowany. Wyniki opisują zatem opóźnienie przed osiągnięciem granicy narzędzia, lecz nie pozwalają wnioskować o maksymalnej przepustowości serwera ani zachowaniu systemu przy setkach równoległych użytkowników. Z tego samego względu przepustowość, przy adaptacyjnym czasie namysłu, pozostaje porównywalna po-

między protokołami wyłącznie w obrębie tej samej komórki macierzy.

Badane scenariusze odzwierciedlają wybrane operacje systemu: generowanie odpowiedzi w pamięci, odczyt katalogu oraz transakcyjny zapis zamówienia. Kontrolowany zbiór danych i stałe kształty komunikatów zwiększają wewnętrzną wiarygodność porównania, ale nie obejmują całego zróżnicowania systemów produkcyjnych, takich jak odmienne rozkłady rozmiarów odpowiedzi, inne strategie buforowania czy dodatkowe warstwy pośredniczące. Przed zastosowaniem wniosków w innym systemie konieczne jest zatem zmierzenie rzeczywistych rozmiarów komunikatów, charakterystyki łącza oraz udziału warstwy danych w czasie odpowiedzi.

Rozdział 8

Porównawcza analiza wydajności REST i gRPC-Web

Wyniki eksperymentu przedstawiono zgodnie z metodologią opisaną w rozdziale 7. Analiza obejmuje wolumen przesyłanych danych, model liczby wymian pakietów, scenariusze Echo, Products i Orders, porównanie konfiguracji komunikacyjnych, granicę przepustowości stanowiska pomiarowego, przedziały ufności oraz weryfikację hipotez.

8.1 Wolumen przesyłanych danych

Rozmiary odpowiedzi wyznaczono pomiarowo na wdrożonym systemie, zestawiając liczbę bajtów zwracanych przez punkt końcowy REST oraz przez usługę gRPC dla identycznego zapytania. Wyniki przedstawia tabela 8.1.

Tabela 8.1: Zmierzony rozmiar odpowiedzi [B] i redukcja uzyskana serializacją binarną

Scenariusz	Rekordów	JSON	Protocol Buffers	Redukcja
Products	10	1 328	752	43,4%
Products	200	26 405	14 998	43,2%
Products	500	66 556	38 054	42,8%
Products	1000	133 134	76 200*	42,8%*
Products	2000	265 159	151 700*	42,8%*
Echo	10	1 771	1 190	32,8%
Echo	200	36 242	24 104	33,5%
* wartość oszacowana przez ekstrapolację zmierzonego współczynnika redukcji				

Serializacja binarna redukuje wolumen przesyłanych danych o 42,8–43,4% w scenariuszu Products oraz o 32,8–33,5% w scenariuszu Echo. Stabilność współczynnika względem liczby rekordów potwierdza jego systematyczny charakter.

Wpływ kompresji treści na wolumen danych

Wszystkie rozmiary przedstawione w tabeli 8.1 dotyczą treści nieskompresowanej. W badanym systemie kompresja ciała odpowiedzi nie została włączona na żadnej ze ścieżek: usługi nie korzystają z oprogramowania pośredniczącego kompresji, a konfiguracja serwera pośredniczącego nie zawiera filtra kompresora. W takim przypadku żaden z badanych wariantów nie jest uprzywilejowany. Odpowiada on przy tym domyślnym ustawieniom platformy ASP.NET Core oraz szkieletu gRPC dla .NET. Konsekwencje tego wyboru dla zakresu wniosków wymagają jednak omówienia, ponieważ w praktyce wdrożeniowej dokumenty JSON podawane są najczęściej z kompresją realizowaną przez serwer aplikacyjny, odwrotne proxy lub sieć dostarczania treści.

Wpływ kompresji oszacowano ilościowo dla scenariusza Echo, którego ładunek generowany jest w sposób deterministyczny. Poprawność odtworzenia zweryfikowano przez porównanie z rozmiarami zmierzonymi na wdrożonym systemie: dla 200 rekordów uzyskano 36 246 B wobec zmierzonych 36 242 B w formacie JSON oraz 24 079 B wobec 24 104 B w formacie binarnym, czyli odchylenie nieprzekraczające 0,1%. Wyniki kompresji algorytmem gzip przedstawia tabela 8.2.

Tabela 8.2: Rozmiar ładunku scenariusza Echo bez kompresji i po kompresji gzip [B] oraz wynikająca z niego liczba okien odbiorczych

Rek.	JSON	JSON gz	protobuf	protobuf gz	Okien JSON gz	Okien protobuf
200	36 246	2 964	24 079	2 523	0,05	0,37
500	91 247	7 129	60 679	5 986	0,11	0,93
2000	370 255	27 715	246 682	22 686	0,42	3,76

Kompresja redukuje wolumen formatu tekstowego 12–13-krotnie, a formatu binarnego 9,5–11-krotnie. Po jej zastosowaniu przewaga objętościowa serializacji binarnej maleje z 33% do 15–18%. Wszystkie badane ładunki mieszczą się wówczas w jednym oknie odbiorczym. Przeniesienie wyników dla transferów nieskompresowanych na wdrożenie z włączoną kompresją wymagałoby odrębnego pomiaru.

8.2 Model liczby wymian pakietów

Zmierzona charakterystyka łącza (sekcja 7.5) ujawniła, że przepustowość pojedynczego połączenia wynosi około 20 Mbit/s, mimo że przepustowość zagregowana przekracza 270 Mbit/s. Iloczyn przepustowości i opóźnienia dla tego połączenia wynosi

$$20 \text{ Mbit/s} \cdot 0,0262 \text{ s} \approx 65\,500 \text{ B} \approx 64 \text{ KiB}, \quad (8.1)$$

co odpowiada rozmiarowi okna odbiorczego protokołu TCP bez zastosowania skalowania okna [33]. Przy oknie mniejszym od iloczynu przepustowości i opóźnienia przepustowość osiągalna w jednym połączeniu wynika z rozmiaru okna, nie z pojemności łącza. Czynnikiem ograniczającym pojedyncze żądanie nie jest zatem pojemność łącza, lecz ilość danych, jaką nadawca może wysłać przed otrzymaniem potwierdzenia: po wypełnieniu okna transmisja wstrzymuje się do nadejścia potwierdzenia od odbiorcy.

Zależność $B = W/RTT$ pozwala oszacować rozmiar okna W na podstawie zmierzonej przepustowości pojedynczego połączenia B i opóźnienia obiegu pakietu. Sama zgodność oszacowania z wartością 64 KiB nie przesądza o tym, czy ograniczeniem jest okno odbiorcze, czy powolny start TCP. Rozróżnienie tych mechanizmów wymagałoby przechwycenia ruchu na poziomie pakietów, którego nie wykonano. Oba mechanizmy przewidują jednak skokowy wzrost kosztu transmisji wraz z przekraczaniem kolejnych wielokrotności rozmiaru okna.

Z zależności tej wynika model przewidujący czas transmisji odpowiedzi. Przesłanie odpowiedzi o rozmiarze s wymaga $\lceil s/64 \text{ KiB} \rceil$ okien, a każde okno ponad pierwsze kosztuje jedno dodatkowe opóźnienie obiegu pakietu. Przewidywana liczba dodatkowych obiegów wynosi zatem

$$n_{\text{RTT}} = \left\lceil \frac{s}{64 \text{ KiB}} \right\rceil - 1. \quad (8.2)$$

Model ten poddano weryfikacji ilościowej, przyjmując za punkt odniesienia konfigurację o 200 rekordach, w której odpowiedź obu formatów mieści się w jednym oknie. Nadwyżkę czasu względem tego punktu odniesienia podzielono przez zmierzone opóźnienie obiegu pakietu (26,2 ms). Wyniki przedstawia tabela 8.3.

Tabela 8.3: Weryfikacja modelu: przewidywana i zmierzona liczba dodatkowych obiegów pakietu

Scenariusz	Rekordów	Rozmiar [B]	Okien	Przewid.	Zmierz.
Echo, JSON	500	90 500*	1,38	1	1,04
Echo, JSON	2000	362 000*	5,52	5	5,15
Echo, protobuf	500	60 500*	0,92	0	0,10
Echo, protobuf	2000	242 000*	3,69	3	3,29
Products, JSON	500	66 556	1,02	1	0,97
Products, JSON	1000	133 134	2,03	2	1,89
Products, JSON	2000	265 159	4,05	4	3,83
Products, protobuf	500	38 054	0,58	0	0,08
Products, protobuf	1000	76 200*	1,16	1	1,06
Products, protobuf	2000	151 700*	2,31	2	2,19
* wartość wyznaczona przez ekstrapolację zmierzonego rozmiaru pojedynczego rekordu					

Zgodność jest bardzo dobra we wszystkich dziesięciu przypadkach: różnica pomiędzy wartością przewidywaną a zmierzoną nie przekracza 0,29 obiegu,

a w żadnym przypadku nie zmienia liczby całkowitej. Model wyjaśnia w szczególności zjawisko, którego nie sposób wyjaśnić samą przepustowością: różnica 30 kilobajtów pomiędzy formatami przy 500 rekordach scenariusza Echo odpowiadałaby przy przepustowości zagregowanej niespełna jednej milisekundy, a przy przepustowości pojedynczego strumienia, około dwunastu milisekund, natomiast zmierzona różnica wynosi 26 ms, czyli dokładnie jeden obieg pakietu.

Przewaga serializacji binarnej nie jest ciągłą funkcją rozmiaru odpowiedzi, lecz realizuje się skokowo: wyłącznie wtedy, gdy mniejszy ładunek wymaga mniejszej liczby okien. Ponadto wielkość tej przewagi jest proporcjonalna do opóźnienia łącza, nie do szybkości serwera: w sieci lokalnej, gdzie opóźnienie obiegu pakietu jest mniejsze od milisekundy, ten sam mechanizm dawałby korzyść pomijalną.

8.3 Analiza narzutu protokołu (Echo)

Scenariusze Echo izolują koszt protokołu od kosztu warstwy danych. Tabela 8.4 przedstawia zmierzone mediany opóźnień wraz ze współczynnikami zmienności.

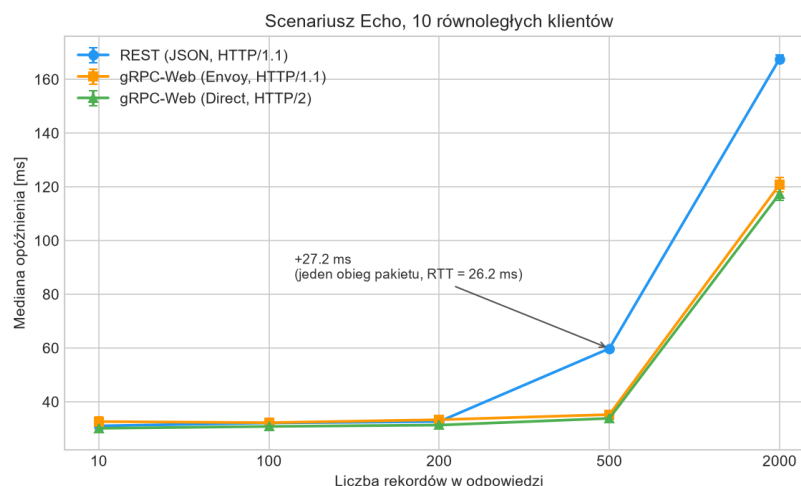
Tabela 8.4: Mediana opóźnienia w scenariuszach Echo [ms]; w nawiasach współczynnik zmienności median pięciu przebiegów

VU	Rekordów	REST	Envoy	Direct	Direct vs REST
10	10	31,0 (0,7%)	32,6 (4,6%)	30,1 (2,9%)	−3%
10	100	32,1 (2,9%)	32,2 (1,5%)	30,8 (3,4%)	−4%
10	200	32,6 (1,1%)	33,3 (1,5%)	31,3 (1,3%)	−4%
10	500	59,8 (1,0%)	35,2 (1,8%)	33,8 (2,3%)	−43%
10	2000	167,6 (0,7%)	120,8 (1,8%)	117,4 (1,6%)	−30%
20	10	37,1 (5,7%)	34,8 (4,0%)	32,5 (3,6%)	−12%
20	100	39,0 (4,3%)	35,6 (6,9%)	33,0 (3,3%)	−15%
20	200	37,0 (5,6%)	34,9 (1,7%)	33,8 (3,0%)	−9%
20	500	60,6 (1,7%)	39,1 (6,0%)	35,5 (11,5%)	−41%

Zależność tę przedstawia rysunek 8.1. Dane pozwalają na sformułowanie trzech obserwacji.

Do 200 rekordów włącznie różnice pomiędzy protokołami pozostają niewielkie, od trzech do piętnastu procent, przy wartościach bezwzględnych rzędu jednej do pięciu milisekund. Zgodnie z modelem z sekcji 8.2 odpowiedzi tego rozmiaru mieszczą się w jednym oknie w obu formatach; liczba obiegów pakietu jest zatem identyczna, a pozostała różnica wynika z narzutu ramkowania i warstwy transportowej.

Przejście z 200 na 500 rekordów powoduje skokową zmianę obrazu. Opóźnienie REST rośnie z 32,6 do 59,8 ms, czyli o 27,2 ms, natomiast opóźnienie wariantu Direct – z 31,3 do 33,8 ms, czyli o 2,5 ms. Przewaga wariantu binarnego osiąga



Rysunek 8.1: Mediana opóźnień w scenariuszach Echo względem liczby rekordów w odpowiedzi

w tej konfiguracji 43%: dokument JSON o rozmiarze 90,5 kilobajta nie mieści się w jednym oknie, natomiast wiadomość binarna o rozmiarze 60,5 kilobajta – mieści.

Przy 2000 rekordach przewaga wynosi 30%, czyli zbliża się do zmierzonego współczynnika redukcji wolumenu danych. Powyżej kilku okien liczba obiegów staje się w przybliżeniu proporcjonalna do rozmiaru odpowiedzi, a przewaga czasowa zbiega wówczas do przewagi objętościowej.

Współczynniki zmienności nie przekraczają w tej grupie 6,9%, z jednym wyjątkiem na poziomie 11,5%. Wszystkie omówione relacje zachodzą konsekwentnie w każdym z pięciu przebiegów.

8.4 Analiza wydajności odczytu (Products)

Scenariusze Products badają wydajność w realistycznym kontekście odczytu katalogu, z pełnym łańcuchem obejmującym pamięć podręczną lub bazę danych, deserializację w przeglądarce oraz aktualizację interfejsu.

Odczyt z ciepłą pamięcią podręczną

Obraz uzyskany w scenariuszu Echo przenosi się na realną ścieżkę odczytu bez istotnych zmian, co ilustruje rysunek 8.2. Do 200 produktów na stronę różnice mieszczą się w przedziale od jednego do szesnastu procent, natomiast od 500 produktów przewaga wariantów binarnych stabilizuje się na poziomie 26–38%. Zgodność z przewidywaniami modelu (tabela 8.3) jest w tej grupie równie dobra jak w scenariuszu Echo.

Wynik ten jest istotny metodycznie, ponieważ scenariusz Products obejmuje wszystkie składniki obecne w docelowym zastosowaniu: dostęp do pamięci pod-

Tabela 8.5: Mediana opóźnień w scenariuszach Products, ciepła pamięć podręczna [ms]; w nawiasach współczynnik zmienności median pięciu przebiegów

VU	PageSize	REST	Envoy	Direct	Direct vs REST
10	10	33,6 (6,7%)	34,9 (2,0%)	31,7 (2,7%)	−6%
10	100	39,2 (8,0%)	41,7 (6,3%)	39,0 (2,6%)	−1%
10	200	40,3 (3,7%)	42,0 (5,7%)	39,0 (5,0%)	−3%
10	500	65,8 (0,6%)	42,3 (4,5%)	41,1 (3,5%)	−38%
10	1000	89,9 (1,4%)	69,4 (4,5%)	66,9 (1,0%)	−26%
10	2000	140,6 (1,6%)	97,5 (1,8%)	96,5 (1,4%)	−31%
20	10	40,9 (4,6%)	37,5 (6,2%)	34,4 (1,4%)	−16%
20	100	45,5 (2,9%)	45,2 (3,0%)	43,6 (7,2%)	−4%
20	200	46,2 (3,0%)	45,9 (3,0%)	44,4 (4,5%)	−4%
20	500	71,8 (1,5%)	48,4 (2,8%)	44,4 (3,1%)	−38%
20	1000	94,3 (2,4%)	70,5 (1,6%)	70,2 (2,2%)	−26%

ręcznej, serializację po stronie serwera, transmisję, deserializację w silniku JavaScript oraz aktualizację drzewa dokumentu. Utrzymanie się przewagi w tych warunkach wskazuje, że nie jest ona artefaktem uproszczonego scenariusza.

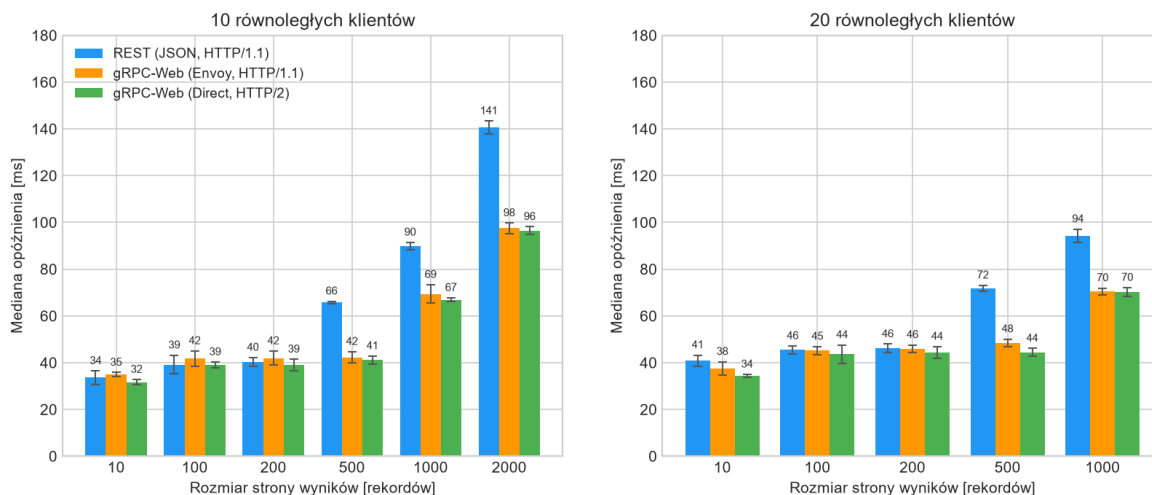
Pomiar przy dwudziestu klientach daje ten sam obraz relacji przy systematycznie wyższych wartościach bezwzględnych. Powtarzalność wyników w tej grupie jest wysoka: współczynnik zmienności nie przekracza ośmiu procent. Charakterystyczne jest przy tym, że najwyższa zmienność występuje przy najmniejszych ładunkach, gdzie mierzona wartość jest zdominowana przez opóźnienie propagacji, a zatem podlega wahaniom warunków sieciowych w większym stopniu względnym.

Odczyt z zimną pamięcią podręczną

Tabela 8.6: Mediana opóźnień w scenariuszach Products, zimna pamięć podręczna [ms]; w nawiasach współczynnik zmienności.

VU	PageSize	REST	Envoy	Direct
10	10	36,8 (51,3%)	38,5 (46,7%)	36,7 (41,4%)
10	100	84,9 (25,7%)	83,8 (24,4%)	80,1 (28,5%)
10	200	82,8 (27,0%)	47,6 (40,9%)	89,9 (4,7%)
10	500	76,7 (14,2%)	55,6 (33,5%)	75,6 (33,2%)
10	1000	100,5 (4,6%)	91,9 (3,5%)	90,0 (11,0%)
10	2000	187,5 (13,5%)	111,3 (7,7%)	116,0 (6,2%)
20	500	89,1 (10,0%)	62,4 (30,9%)	78,3 (21,6%)
20	1000	106,3 (8,9%)	109,2 (10,8%)	111,3 (17,7%)

Grupa ta wymaga ostrożnej interpretacji. Wymuszenie zapytania do bazy danych przy każdym żądaniu zwiększa nie tylko opóźnienie, lecz przede wszystkim jego zmienność: współczynnik zmienności osiąga w skrajnych przypadkach 51%,



Rysunek 8.2: Mediana opóźnienia w scenariuszach Products z ciepłą pamięcią podręczną, dla dziesięciu i dwudziestu równoległych klientów; słupki błędów odpowiadają 95% przedziałowi ufności wartości centralnej.

wobec kilku procent w scenariuszach z ciepłą pamięcią podręczną. W konsekwencji przy małych stronach wyników kolejność protokołów jest niestabilna: w konfiguracji o 200 produktach zmierzone mediany wynoszą 82,8, 47,6 i 89,9 ms, a uporządkowanie to nie powtarza się w konfiguracji o 10 produktach. Różnice te mieszczą się w granicach zmienności tej grupy i nie pozwalają wskazać szybszego protokołu. Zależność tę wraz z przedziałami ufności wartości centralnej przedstawia rysunek 8.3.

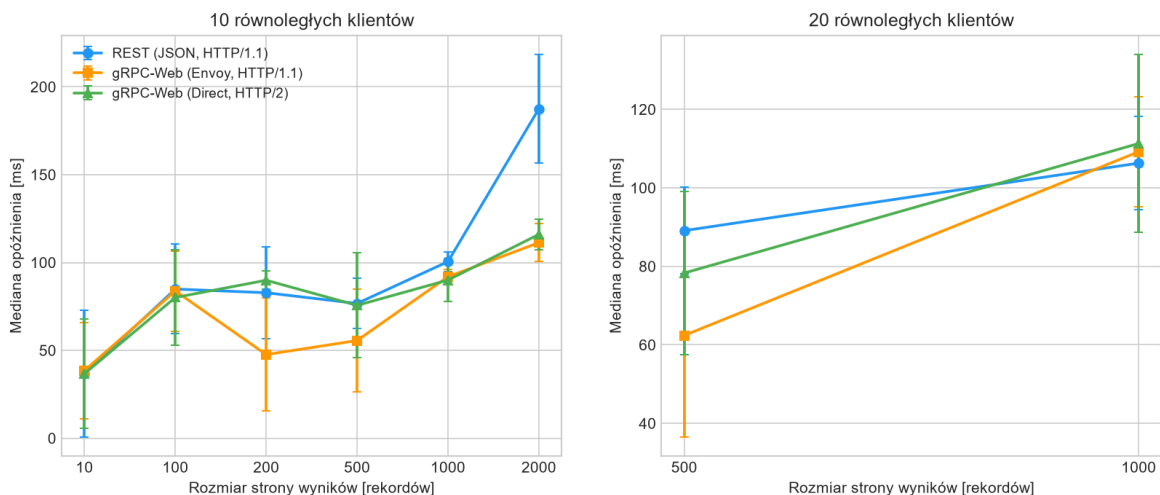
Przy dużych stronach wyników przewaga wariantów binarnych przewyższa poziom szumu. Przy 2000 produktach REST osiąga 187,5 ms, wariant z proxy 111,3 ms, a wariant bezpośredni 116,0 ms, przy współczynniku zmienności nieprzekraczającym 13,5%.

Przy małych ładunkach zmienność wynikająca z zimnej pamięci podręcznej przewyższa różnicę wynikającą z protokołu, przez co ta grupa nie stanowi użytecznego narzędzia porównawczego. Do izolowania wpływu protokołu służy scenariusz Echo, natomiast do oceny zachowania w warunkach realnych – scenariusz z ciepłą pamięcią podręczną.

8.5 Analiza wydajności zapisu (Orders)

Scenariusze Orders angażują pełen łańcuch transakcyjny: walidację oraz zapis encji zamówienia wraz z pozycjami.

Scenariusze zapisu nie wykazują spójnej różnicy pomiędzy protokołami. Przy dziesięciu i pięćdziesięciu pozycjach zamówienia mediany mieszczą się w przedziale 83,5–165,8 ms. Górna wartość dotyczy wariantu Direct przy pięćdziesięciu pozycjach i dwudziestu klientach; pozostałe mediany w tej grupie wynoszą 83,5–



Rysunek 8.3: Mediana opóźnień w scenariuszach Products z zimną pamięcią podręczną; słupki błędów odpowiadają 95% przedziałowi ufności wartości centralnej

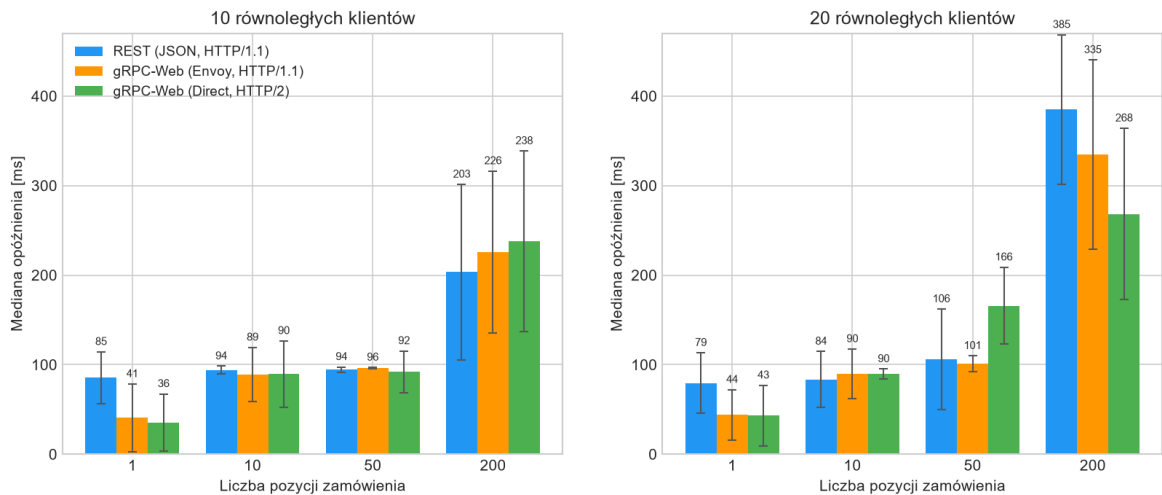
Tabela 8.7: Mediana opóźnień w scenariuszach Orders [ms]; w nawiasach współczynnik zmienności

VU	Pozycji	REST	Envoy	Direct
10	1	85,4 (29,9%)	40,6 (50,7%)	35,6 (54,8%)
10	10	94,2 (3,5%)	89,1 (29,9%)	89,6 (40,3%)
10	50	94,5 (2,5%)	96,1 (0,7%)	91,8 (22,0%)
10	200	203,4 (39,1%)	225,9 (32,3%)	237,8 (32,1%)
20	1	79,3 (39,8%)	43,8 (40,1%)	43,1 (47,9%)
20	10	83,5 (35,0%)	89,9 (26,7%)	89,7 (4,9%)
20	50	106,2 (40,0%)	101,2 (6,9%)	165,8 (21,6%)
20	200	384,9 (16,6%)	334,7 (28,3%)	268,2 (28,4%)

106,2 ms. Przy dwustu pozycjach kolejność jest odmienna dla dwóch badanych poziomów obciążenia, a współczynniki zmienności sięgają czterdziestu procent. Ze-stawienie median wraz z przedziałami ufności wartości centralnej przedstawia rysunek 8.4.

Wyjaśnienie wyniku bezpośrednio z modelu przedstawionego w sekcji 8.2 oraz z charakteru ścieżki zapisu. Żądania w tym scenariuszu są niewielkie: nawet dwieście pozycji zamówienia to około siedemnastu kilobajtów, czyli znacznie mniej niż okno odbiorcze. Liczba obiegów pakietu jest zatem identyczna dla wszystkich formatów, a mechanizm dający przewagę w odczycie nie ma tu zastosowania. Jednocześnie czas odpowiedzi zdominowany jest przez stały koszt serwerowy: transakcję bazodanową obejmującą zapis encji.

W badanych scenariuszach zapisu nie uzyskano stałej przewagi jednego wariantu. Szerokie przedziały ufności ograniczają rozstrzygalność różnic; wynik ten nie dowodzi równoważności protokołów ani braku wpływu sposobu komunikacji na czas



Rysunek 8.4: Mediana opóźnień w scenariuszach Orders względem liczby pozycji zamówienia; słupki błędów odpowiadają 95% przedziałowi ufności wartości centralnej

operacji.

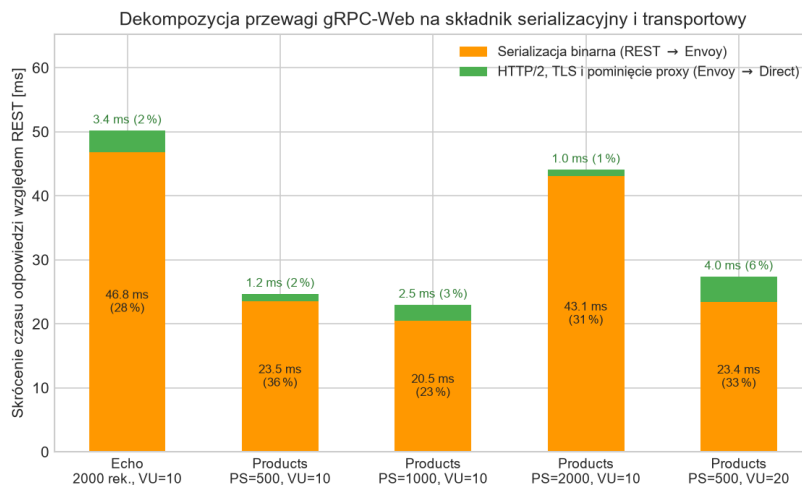
8.6 Porównanie konfiguracji komunikacyjnych

Ścieżka bezpośrednia korzysta z HTTP/2 i TLS, a badany wariant REST z HTTP/1.1. Wariant z Envoy przenosi ładunek binarny, używając HTTP/1.1 na odcinku pomiędzy przeglądarką a proxy. Różnica REST–Envoy obejmuje obecność proxy i odmienny sposób obsługi komunikatów. Różnica Envoy–Direct obejmuje natomiast jednocześnie pominięcie proxy, TLS oraz zmianę wersji HTTP. Zestawienie nie wyodrębnia przyczynowego udziału każdego z tych czynników.

Tabela 8.8: Czasy konfiguracji i różnice względem mediany REST

Konfiguracja	REST	Envoy	Direct	Envoy–REST	Direct–Envoy
Echo, 2000 rek., VU=10	167,6	120,8	117,4	–28%	–2%
Products, PS=500, VU=10	65,8	42,3	41,1	–36%	–2%
Products, PS=1000, VU=10	89,9	69,4	66,9	–23%	–3%
Products, PS=2000, VU=10	140,6	97,5	96,5	–31%	–1%
Products, PS=500, VU=20	71,8	48,4	44,4	–33%	–6%

W konfiguracji Products, PS=500 i VU=10 różnica REST–Direct wynosi 24,7 ms. Z tej wartości 23,5 ms stanowi różnicę REST–Envoy, a 1,2 ms różnicę Envoy–Direct. Różnica REST–Envoy odpowiada zatem około 95% całkowitej różnicy REST–Direct w tej konkretnej konfiguracji. Wynik wskazuje na znaczenie formatu i wolumenu odpowiedzi, lecz nie pozwala wyznaczyć liczbowego udziału samej serializacji.



Rysunek 8.5: Różnice czasów trzech konfiguracji względem mediany opóźnienia REST; wartości procentowe odniesiono do mediany REST

8.7 Skalowanie z liczbą klientów i granica stanowiska pomiarowego

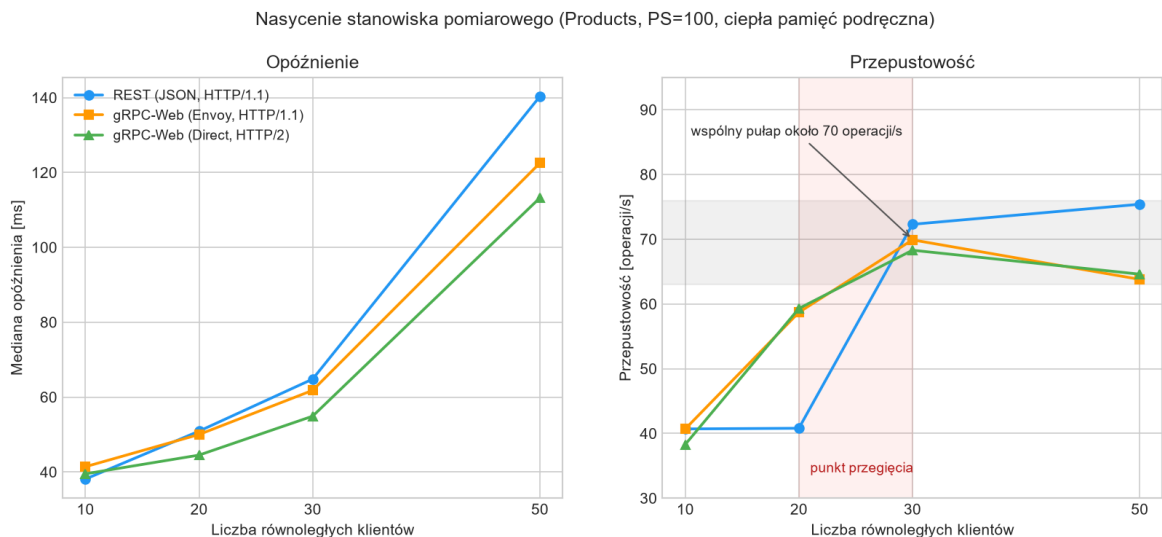
Zakres badanych poziomów obciążenia wyznaczono na podstawie odrębnego pomiaru, którego celem było ustalenie, do jakiej liczby równoległych klientów stanowisko pomiarowe pozostaje wiarygodne. Wyniki dla konfiguracji o 100 produktach na stronę i ciepłej pamięci podręcznej przedstawia tabela 8.9.

Tabela 8.9: Skalowanie z liczbą równoległych klientów: mediana opóźnienia [ms] i przepustowość [operacji/s]

VU	Mediana opóźnienia [ms]			Przepustowość [oper./s]		
	REST	Envoy	Direct	REST	Envoy	Direct
10	38,1	41,4	39,5	40,7	40,7	38,3
20	50,9	50,0	44,5	40,8	58,7	59,3
30	64,8	61,8	54,9	72,3	69,9	68,3
50	140,3	122,5	113,2	75,4	63,8	64,6

Zestawienie, zilustrowane na rysunku 8.6, ujawnia klasyczny obraz nasycenia. Przepustowość wypłaszcza się na poziomie około siedemdziesięciu operacji na sekundę, osiągając wartość graniczną pomiędzy dwudziestoma a trzydziestoma klientami, natomiast opóźnienie nadal narasta, z 38 do 140 ms. Powyżej punktu przegięcia zwiększanie współbieżności wydłuża wyłącznie czas oczekiwania w kolejce, bez korzyści przepustowościowej.

W odczycie z ciepłą pamięcią podręczną REST zwraca gotowy dokument JSON z Redis, natomiast oba warianty gRPC odtwarzają obiekt z bajtów Protocol Buffers i ponownie go serializują. Zbliżony pułap przepustowości mimo tych różnic jest przesłanką ograniczenia po stronie wspólnego klienta pomiarowego, na przy-



Rysunek 8.6: Opóźnienie i przepustowość względem liczby równoległych klientów; szary pas oznacza wspólny pułap przepustowości, a różowy położenie punktu przegięcia

kład rywalizacji o zasoby pomiędzy kontekstami przeglądarki. Samo porównanie ścieżek nie wyklucza jednak wspólnego ograniczenia po stronie serwera lub sieci; lokalizacja wąskiego gardła pozostaje wnioskiem pośrednim.

Wniosek ten stanowił podstawę ograniczenia macierzy badawczej do dziesięciu i dwudziestu klientów, to jest poniżej punktu przegięcia. Przyjęto zasadę, że węższy zakres współbieżności z wynikami wiarygodnymi jest wartościowszy niż zakres szerszy z wynikami opisującymi ograniczenia narzędzia.

8.8 Powtarzalność pomiarów i przedziały ufności

Pięciokrotne wykonanie całej macierzy umożliwia ocenę powtarzalności. Tabela 8.10 przedstawia rozkład współczynnika zmienności median przebiegów w poszczególnych grupach scenariuszy.

Tabela 8.10: Współczynnik zmienności median pięciu przebiegów, w podziale na grupy scenariuszy

Grupa scenariuszy	Minimum	Typowo	Maksimum
Echo	0,7%	1,5–5%	11,5%
Products, ciepła pamięć	0,6%	2–6%	8,0%
Products, zimna pamięć	3,5%	10–30%	51,3%
Orders	0,7%	20–40%	54,8%

Rozkład ten układa się w interpretowalny wzorzec, zgodny z udziałem warstwy danych w czasie odpowiedzi. Najwyższą powtarzalność wykazują scenariusze Echo, nieodwołujące się do bazy ani pamięci podręcznej, oraz scenariusze odczytu

z ciepłą pamięcią podręczną. Scenariusze wymuszające zapytanie do bazy oraz scenariusze zapisu charakteryzują się zmiennością o rząd wielkości wyższą, ponieważ angażują mechanizmy trwałego utrwalania danych, dziennik transakcji oraz blokady wierszy.

Przedziały ufności i rozstrzygalność zmierzonych różnic

Zestawienia przedstawione w poprzednich sekcjach podają wartości centralne wraz z miarą rozrzutu. Ocena, czy zaobserwowana różnica jest rzeczywista, wymaga natomiast miary niepewności samej wartości centralnej.

Tabela 8.11: Przedziały ufności 95% dla wartości centralnej opóźnienia [ms]; $k = 5$ przebiegów, $t = 2,776$

Konfiguracja	REST	Envoy	Direct
Echo, 200 rek., VU=10	$32,5 \pm 0,4$	$33,4 \pm 0,6$	$31,4 \pm 0,5$
Echo, 500 rek., VU=10	$59,7 \pm 0,8$	$35,2 \pm 0,8$	$34,0 \pm 1,0$
Echo, 2000 rek., VU=10	$168,0 \pm 1,5$	$120,4 \pm 2,7$	$117,5 \pm 2,4$
Echo, 500 rek., VU=20	$60,6 \pm 1,3$	$39,7 \pm 3,0$	$37,0 \pm 5,3$
Products, PS=200, VU=10	$40,6 \pm 1,9$	$42,0 \pm 3,0$	$39,2 \pm 2,4$
Products, PS=500, VU=10	$65,8 \pm 0,5$	$42,3 \pm 2,4$	$40,8 \pm 1,8$
Products, PS=1000, VU=10	$90,0 \pm 1,5$	$69,9 \pm 3,9$	$66,9 \pm 0,8$
Products, PS=2000, VU=10	$140,8 \pm 2,7$	$97,5 \pm 2,2$	$96,8 \pm 1,6$
Products, PS=500, VU=20	$71,9 \pm 1,3$	$48,7 \pm 1,7$	$44,4 \pm 1,7$
Products, PS=1000, VU=20	$94,8 \pm 2,8$	$70,7 \pm 1,4$	$70,3 \pm 1,9$
Orders, 10 poz., VU=10	$93,2 \pm 4,1$	$81,3 \pm 30,2$	$73,7 \pm 36,9$
Orders, 50 poz., VU=10	$93,5 \pm 3,0$	$96,0 \pm 0,8$	$86,2 \pm 23,5$
Orders, 10 poz., VU=20	$72,0 \pm 31,3$	$83,0 \pm 27,5$	$89,1 \pm 5,4$
Orders, 50 poz., VU=20	$112,7 \pm 55,9$	$101,2 \pm 8,6$	$160,3 \pm 43,0$

W scenariuszach Echo oraz w odczycie z ciepłą pamięcią podręczną połowkowa szerokość przedziału nie przekracza kilku milisekund, natomiast w scenariuszach zapisu sięga kilkudziesięciu, przekraczając w skrajnym przypadku połowę samej wartości centralnej.

Zasadnicze znaczenie ma jednak zestawienie różnic pomiędzy protokołami w układzie sparowanym. Wartość dodatnia oznacza, że pierwszy z wymienionych protokołów jest wolniejszy. Kolejne akapity omawiają wyniki dla dużych i małych odpowiedzi, wariantów gRPC-Web oraz scenariuszy zapisu.

W każdej konfiguracji odczytu od 500 rekordów przewaga wariantów binarnych jest rozstrzygnięta z dużym marginesem: przedział dla różnicy REST–Direct wynosi na przykład $24,9 \pm 1,6$ ms przy 500 produktach oraz $50,5 \pm 3,3$ ms przy 2000 rekordach scenariusza Echo.

W zakresie małych ładunków obraz jest bardziej złożony, niż wskazywałaby sama wielkość różnicy. W scenariuszu Products przy 200 produktach różnica REST–

Tabela 8.12: Przedziały ufności 95% dla różnic sparowanych w obrębie przebiegu [ms]; wartość dodatnia oznacza dłuższy czas odpowiedzi pierwszego z protokołów

Konfiguracja	REST–Envoy	REST–Direct	Envoy–Direct	Rozstrz.
Echo, 200 rek., VU=10	$-0,9 \pm 0,9$	$1,1 \pm 0,7$	$2,0 \pm 0,3$	tak
Echo, 500 rek., VU=10	$24,5 \pm 1,0$	$25,8 \pm 1,6$	$1,2 \pm 0,9$	tak
Echo, 2000 rek., VU=10	$47,6 \pm 3,8$	$50,5 \pm 3,3$	$2,9 \pm 3,4$	tak
Echo, 500 rek., VU=20	$20,9 \pm 2,7$	$23,6 \pm 5,7$	$2,7 \pm 7,2$	tak
Products, PS=200, VU=10	$-1,4 \pm 3,8$	$1,4 \pm 2,5$	$2,7 \pm 2,4$	nie
Products, PS=500, VU=10	$23,4 \pm 2,4$	$24,9 \pm 1,6$	$1,5 \pm 1,4$	tak
Products, PS=1000, VU=10	$20,1 \pm 4,3$	$23,1 \pm 1,2$	$3,0 \pm 3,7$	tak
Products, PS=2000, VU=10	$43,3 \pm 4,9$	$44,0 \pm 3,5$	$0,8 \pm 2,2$	tak
Products, PS=500, VU=20	$23,2 \pm 1,4$	$27,4 \pm 1,5$	$4,2 \pm 1,8$	tak
Products, PS=1000, VU=20	$24,1 \pm 3,6$	$24,5 \pm 1,9$	$0,4 \pm 3,1$	tak
Orders, 10 poz., VU=10	$12,0 \pm 32,5$	$19,5 \pm 35,5$	$7,6 \pm 31,9$	nie
Orders, 50 poz., VU=10	$-2,4 \pm 3,1$	$7,4 \pm 25,4$	$9,8 \pm 23,8$	nie
Orders, 10 poz., VU=20	$-11,0 \pm 21,3$	$-17,1 \pm 28,9$	$-6,1 \pm 22,6$	nie
Orders, 50 poz., VU=20	$11,5 \pm 60,1$	$-47,6 \pm 76,0$	$-59,1 \pm 45,2$	nie
Kolumna <i>Rozstrz.</i> odnosi się do różnicy REST–Direct				

Direct wynosi $1,4 \pm 2,5$ ms, a zatem nie jest rozstrzygnięta. W scenariuszu Echo przy tej samej liczbie rekordów różnica wynosi $1,1 \pm 0,7$ ms i pozostaje bez znaczenia praktycznego wobec opóźnienia obiegu pakietu wynoszącego 26,2 ms.

Różnica pomiędzy oboma wariantami gRPC-Web jest rozstrzygnięta jedynie w pięciu z dziesięciu konfiguracji odczytu, a jej wielkość mieści się w przedziale od $0,4 \pm 3,1$ do $4,2 \pm 1,8$ ms. Wielkość efektu jest zatem porównywalna z niepewnością jego oszacowania, co uzasadnia wniosek o drugorzędnym znaczeniu wyboru pomiędzy wariantem bezpośrednim a wariantem z proxy.

W scenariuszach zapisu żadna z różnic nie jest rozstrzygnięta. Przedziały są w tej grupie od kilku do kilkudziesięciu razy szersze niż w odczycie. Jedyny wyjątek dotyczy pary Envoy–Direct przy pięćdziesięciu pozycjach i dwudziestu klientach ($-59,1 \pm 45,2$ ms), lecz znak tej różnicy jest przeciwny niż w pozostałych komórkach tej grupy, a jej wartość wynika z jednego odstającego przebiegu.

8.9 Weryfikacja hipotez badawczych

H1 (warianty gRPC-Web osiągają niższe opóźnienie niż REST, a przewaga rośnie z rozmiarem odpowiedzi): hipoteza potwierdzona z zastrzeżeniem dotyczącym charakteru zależności. Przewaga występuje i jest znaczna: 26–43% przy odpowiedziach od 500 rekordów. Wzrost nie ma charakteru ciągłego. Efekt ujawnia się po przekroczeniu rozmiaru okna odbiorczego, natomiast w zakresie do 200 rekordów pozostaje w granicach kilku procent.

H2 (przy małych odpowiedziach różnice są nieistotne): hipoteza potwier-

dzona. Przy odpowiedziach do 200 rekordów różnice wynoszą od jednego do szesnastu procent, przy wartościach bezwzględnych od jednej do pięciu milisekund, wobec opóźnienia obiegu pakietu wynoszącego 26,2 ms. Samo opóźnienie propagacji stanowi w tym zakresie od około pięćdziesięciu siedmiu procent czasu odpowiedzi przy dwudziestu równoległych klientach i dwustu rekordach do siedemdziesięciu ośmiu procent przy dziesięciu klientach i dziesięciu rekordach. W tym zakresie różnice są praktycznie nieistotne.

H3 (wariant gRPC-Web przez Envoy zachowuje przewagę względem REST przy HTTP/1.1 na odcinku przeglądawkowym): hipoteza potwierdzona w zakresie badanych konfiguracji. W konfiguracji Products, PS=500 i VU=10 różnica REST–Envoy stanowi około 95% różnicy REST–Direct. Wynik wskazuje na znaczenie formatu i wolumenu odpowiedzi, jednak zastosowany układ nie pozwala oddzielnie oszacować wpływu serializacji, TLS, proxy i wersji HTTP.

H4 (w operacjach zapisu różnice zanikają): nie wykryto stałej przewagi jednego wariantu, ale wyniki nie wystarczają do potwierdzenia równoważności protokołów. Przy dziesięciu i pięćdziesięciu pozycjach zamówienia mediany wynoszą 83,5–165,8 ms. Przedziały ufności dla różnic REST–Direct obejmują zero, a dla pary Envoy–Direct występuje wyjątek omówiony w sekcji 8.8. Udział transakcji bazodanowej i zmienność wyników ograniczają możliwość rozstrzygnięcia hipotezy.

H5 (wariant Direct osiąga niższe opóźnienie niż wariant z proxy Envoy): wyniki częściowo wspierają hipotezę. Wariant bezpośredni ma niższą medianę w każdej konfiguracji odczytu z ciepłą pamięcią podręczną oraz scenariusza Echo. Skrócenie czasu wynosi około 0,4–9,2%, licząc względem mediany Envoy jako $(T_{\text{Envoy}} - T_{\text{Direct}})/T_{\text{Envoy}}$. Nie wszystkie różnice są jednak rozstrzygnięte statystycznie. W grupie scenariuszy z zimną pamięcią podręczną relacja ta nie zachodzi konsekwentnie. Wybór wariantu wdrożenia wymaga uwzględnienia również topologii systemu.

8.10 Rekomendacje i wytyczne wdrożeniowe

Uzyskane wyniki nie uzasadniają bezwarunkowego zastąpienia REST przez gRPC-Web. Wskazują natomiast mierzalne kryteria, na podstawie których można dobrać mechanizm komunikacji do charakterystyki konkretnego systemu. Decyzja powinna wynikać z rozmiaru przesyłanych danych, parametrów łącza, dominującego rodzaju operacji oraz wymagań eksploatacyjnych.

Rekomendacje architektoniczne

1. **Rozmiar odpowiedzi należy oceniać względem efektywnego okna transmisji.** Zastosowanie formatu binarnego przynosi istotną korzyść wtedy, gdy

zmniejszenie wolumenu ogranicza liczbę wymian pakietów. W badanym środowisku efekt pojawiał się przy nieskompresowanych odpowiedziach od około 500 rekordów i osiągał 26–43%. Dla małych odpowiedzi różnice pozostawały na poziomie pojedynczych milisekund, dlatego sam wybór gRPC-Web nie powinien być traktowany jako uniwersalna optymalizacja.

2. **Przed zmianą protokołu należy zmierzyć ładunek po kompresji.** Kompresja gzip zmniejszyła w oszacowaniu przewagę objętościową Protocol Buffers z około 33% do 15–18% i sprowadziła wszystkie analizowane odpowiedzi poniżej rozmiaru jednego okna. W systemach podających dokumenty JSON z kompresją właściwym pierwszym krokiem jest zatem pomiar rzeczywistego rozmiaru odpowiedzi, a następnie test opóźnienia w docelowej sieci. Jeżeli ładunek nadal przekracza próg jednego okna, należy rozważyć również paginację, ograniczenie pól odpowiedzi lub zmianę sposobu agregacji danych.
3. **Charakter operacji jest ważniejszy niż sam format komunikatu.** W odczycie dużych zbiorów koszt transmisji może dominować i uzasadniać zastosowanie gRPC-Web. W operacjach zapisu angażujących transakcję bazodanową różnice pomiędzy protokołami nie były rozstrzygalne, ponieważ czas odpowiedzi wyznaczała warstwa persystencji. W takich ścieżkach priorytetem powinny być optymalizacja zapytań, zakres transakcji i organizacja zapisu, natomiast protokół należy dobierać ze względu na kontrakt, interoperacyjność i koszt utrzymania.
4. **Wybór pomiędzy wariantem bezpośrednim a proxy powinien wynikać z topologii systemu.** W scenariuszach Echo i odczytu z ciepłą pamięcią podręczną Direct skracał medianę czasu względem Envoy o około 0,4–9,2%, przy czym nie wszystkie różnice były rozstrzygnięte statystycznie. Obsługa bezpośrednia upraszcza ścieżkę żądania i ogranicza liczbę komponentów, natomiast serwer pośredniczący może być uzasadniony, gdy architektura wymaga wspólnej warstwy routingu i centralnej konfiguracji ruchu. Nie należy rezygnować z proxy wyłącznie na podstawie oczekiwania znaczącego skrócenia czasu odpowiedzi.
5. **Pamięć podręczna powinna być traktowana również jako mechanizm stabilizacji czasu odpowiedzi.** W scenariuszach z ciepłą pamięcią podręczną współczynnik zmienności wynosił typowo 2–6%, natomiast przy wymuszonym dostępie do bazy wzrastał do 10–30%, a maksymalnie do 51,3%. Cache-Aside poprawia więc nie tylko wartość centralną opóźnienia, lecz także jego przewidywalność. Wymaga to jednocześnie jawnej polityki unieważniania danych oraz osobnego monitorowania ścieżek trafienia i chybienia.

Wytyczne wdrożeniowe

1. **Test wydajnościowy należy prowadzić w warunkach odpowiadających docelowemu wdrożeniu.** Powinien on obejmować rzeczywistą przeglądarkę, końcową konfigurację TLS, kompresji i serwerów pośredniczących oraz reprezentatywne opóźnienie sieci. Pomiar wyłącznie wewnątrz centrum danych może ukryć efekt zależny od liczby obiegów pakietu, natomiast test bez kompresji może zawyżyć przewagę formatu binarnego względem konfiguracji produkcyjnej.
2. **Należy oddzielić ograniczenia systemu od ograniczeń generatora obciążenia.** Wspólny pułap przepustowości wszystkich protokołów ujawnił w badaniu nasycenie stanowiska klienckiego. Przed interpretacją wyników trzeba więc wykonać pomiar skalowania i wyznaczyć punkt przegięcia. Badania większej współbieżności powinny wykorzystywać wiele maszyn generujących ruch, przy zachowaniu pomiaru czasu wewnątrz strony i kontroli obciążenia każdej z nich.
3. **Porównywane ścieżki muszą zachowywać symetrię funkcjonalną.** Powinny wykonywać tę samą logikę biznesową, korzystać z tych samych danych i kończyć pomiar w tym samym punkcie cyklu renderowania. Należy też pomiarowo potwierdzić negocjowaną wersję HTTP, liczbę połączeń, rozmiar odpowiedzi i stan pamięci podręcznej. Bez tej kontroli wynik może opisywać różnicę implementacji, a nie protokołu.
4. **Warstwę obserwowalności należy wykorzystać do weryfikacji przyczyn, nie jako zamiennik pomiaru klienckiego.** Metryki serwerowe powinny obejmować co najmniej czas obsługi żądania, liczbę żądań, wykorzystanie zasobów oraz zachowanie środowiska uruchomieniowego. Ich zestawienie z czasem mierzonym w przeglądarce pozwala rozróżnić koszt przetwarzania po stronie usługi od transmisji, deserializacji i aktualizacji interfejsu.
5. **Decyzję protokołową należy okresowo ponawiać po zmianach systemu.** Zmiana modelu odpowiedzi, poziomu kompresji, lokalizacji użytkowników, konfiguracji proxy lub sposobu korzystania z pamięci podręcznej może przesunąć granicę, przy której format binarny daje korzyść. Kryterium wdrożeniowym powinien być zatem wynik powtarzalnego testu dla reprezentatywnych scenariuszy, a nie właściwość przypisana protokołowi niezależnie od kontekstu.

Rozdział 9

Podsumowanie

W ramach niniejszej pracy:

- zebrano i usystematyzowano wiedzę z zakresu architektury mikroserwisowej, komunikacji synchronicznej oraz wzorców projektowych dla systemów rozproszonych w ekosystemie .NET,
- dokonano szczegółowej analizy protokołu gRPC-Web i jego wariantów wdrożeniowych (proxy Envoy, bezpośrednia obsługa w oprogramowaniu pośredniczącym ASP.NET Core),
- zaprojektowano i zaimplementowano autorski system testowy składający się z dwóch mikroserwisów .NET 10 (ProductService, OrderService), infrastruktury wspierającej (PostgreSQL, Redis, Envoy Proxy) oraz warstwy obserwowalności (Prometheus, Grafana),
- zaimplementowano wzorzec Cache-Aside z wykorzystaniem Redis, a także skonfigurowano serwer Kestrel do równoległej obsługi trzech protokołów,
- opracowano metodykę pomiarową opartą na rzeczywistej przeglądarce internetowej, w której czas mierzony jest wewnątrz strony, od interakcji użytkownika do naniesienia odpowiedzi na drzewo dokumentu, co obejmuje pełną ścieżkę odczuwaną przez użytkownika aplikacji przeglądarkowej,
- przeprowadzono 39 konfiguracji pomiarowych w pięciu niezależnych powtórzeniach z rotowaną kolejnością protokołów, co dało 585 wykonanych testów i 171 tysięcy pomiarów, badając wpływ protokołu komunikacji, rozmiaru odpowiedzi, stanu pamięci podręcznej oraz liczby równoległych klientów na czas odpowiedzi systemu,
- poddano wyniki analizie statystycznej, wyznaczając przedziały ufności dla różnic sparowanych w obrębie przebiegu.

Główne wnioski z przeprowadzonych badań eksperymentalnych:

1. Warianty gRPC-Web skracają czas odpowiedzi o 26–43% względem REST dla dużych, nieskompresowanych odpowiedzi. Efekt pojawiał się po przekroczeniu zaobserwowanego w badanym środowisku progu około 64 KiB. Poniżej tego progu różnice wynosiły kilka procent.
2. W analizie objętościowej gzip zmniejszył dokument JSON dwunasto- do trzy-nastokrotnie, a wiadomość binarną około dziesięciokrotnie. Przewaga objętościowa formatu binarnego spadła z 33% do 15–18%, a skompresowane ła-dunki nie przekroczyły progu 64 KiB. Czasu odpowiedzi z włączoną kompresją nie mierzono.
3. Zależność przewagi od rozmiaru odpowiedzi miała charakter skokowy. Model oparty na efektywnym progu transmisji około 64 KiB odtworzył zmierzone przy-rosty czasu w dziesięciu konfiguracjach z dokładnością do 0,29 RTT.
4. W konfiguracji Products, PS=500 i VU=10 różnica REST–Envoy stanowiła około 95% różnicy REST–Direct. Wariant przez Envoy uzyskał przewagę nad REST także przy HTTP/1.1 na odcinku przeglądarkowym. Wynik wskazuje na znaczenie formatu i wolumenu odpowiedzi, lecz nie wyodrębnia liczbowego udziału serializacji, TLS, proxy ani wersji HTTP.
5. Serializacja Protocol Buffers redukuje wolumen przesyłanych danych o 32–43%. Wielkość ta nie zależy od warunków sieciowych, obciążenia ani od za-stosowanego narzędzia pomiarowego. Jedynym czynnikiem, który ją zmienia, jest kompresja treści omówiona we wniosku drugim.
6. Przewaga protokołów binarnych skaluje się z opóźnieniem sieci, a nie z szyb-kością serwera, ponieważ wyraża się liczbą zaoszczędzonych obiegów pa-kietu. W komunikacji przez łącze rozległe o opóźnieniu 26 ms jest znacząca, natomiast w sieci lokalnej stałaby się pomijalna.
7. W operacjach zapisu nie wykryto stałej przewagi jednego wariantu. Dla dzie-sięciu i pięćdziesięciu pozycji zamówienia mediany wynosiły 83,5–165,8 ms. Przedziały ufności dla różnic REST–Direct obejmowały zero, co nie stanowi do-wodu równoważności protokołów. Interpretację ograniczała zmienność zwią-zana z obsługą transakcji bazodanowej.
8. W scenariuszach Echo i odczytu z ciepłą pamięcią podręczną wariant Direct skracał medianę czasu o około 0,4–9,2% względem Envoy. Nie wszystkie róż-nice były rozstrzygnięte statystycznie, a przy zimnej pamięci podręcznej kolej-ność wariantów nie była stała.
9. Zmienność pomiarów okazała się silnie zależna od udziału warstwy danych w czasie odpowiedzi: od poniżej jednego procenta w scenariuszach izolują-

cych narzut protokołu do ponad pięćdziesięciu procent w scenariuszach wymuszających zapytanie do bazy oraz w operacjach zapisu. Wprowadzenie pamięci podręcznej obniża tę zmienność.

10. Przepustowość stanowiska pomiarowego przestawała rosnąć pomiędzy dwudziestoma a trzydziestoma równoległymi klientami, podczas gdy opóźnienie nadal wzrastało. Wspólny pułap trzech wariantów wskazywał na ograniczenie po stronie klienta. Z tego powodu główna macierz obejmowała najwyżej dwudziestu klientów.

W ramach niniejszej pracy zaprojektowano i zrealizowano działający system mikroserwisowy oraz przeprowadzono na nim zaplanowany zestaw badań. Wyniki określają relację wydajnościową badanych konfiguracji w zależności od rozmiaru odpowiedzi względem okna odbiorczego protokołu TCP. Wszystkie pięć hipotez poddano weryfikacji, przy czym w przypadku H1 doprecyzowano charakter zależności od rozmiaru odpowiedzi, a w przypadku H5 praktyczną istotność wykazanej różnicy.

Uzyskane wyniki dotyczą badanych konfiguracji komunikacyjnych. Korzyść wariantów gRPC-Web pojawiała się wtedy, gdy mniejszy wolumen odpowiedzi ograniczał liczbę wymian pakietów potrzebnych do jej przesłania. Przy małych ładunkach, w operacjach zapisu oraz po zastosowaniu kompresji różnice pozostawały praktycznie nieistotne.

Zaprojektowany system charakteryzują następujące cechy:

- równoległa obsługa trzech protokołów komunikacji w ramach jednej aplikacji klienckiej, umożliwiająca ich porównanie przy identycznej logice biznesowej, tych samych danych oraz niezmienionej ścieżce prezentacji,
- podejście Contract-First, w którym pliki `.proto` stanowią jedyne źródło prawdy, a kod serwera (C#) i klienta (TypeScript) jest z nich generowany,
- warstwa obserwowalności zgodna ze standardem OpenTelemetry, obejmująca metryki eksportowane do Prometheusa i pulpity wizualizacyjne w Grafanie, umożliwiającą korelację wyników eksperymentu z wewnętrznym stanem systemu.

Kierunki dalszego rozwoju pracy obejmują:

- rozdzielenie stanowiska pomiarowego na wiele maszyn klienckich, co pozwoliłoby przenieść granicę wiarygodności opisaną w ostatnim z powyższych wniosków i objąć pomiarem poziomy współbieżności rzędu setek równoległych klientów,
- powtórzenie eksperymentu przy różnych wartościach opóźnienia łącza, co umożliwiłoby empiryczną weryfikację przewidywanej proporcjonalności przewagi protokołów binarnych do opóźnienia sieci,

- powtórzenie macierzy z włączoną kompresją ciała odpowiedzi na wszystkich ścieżkach, co pozwoliłoby zweryfikować pomiarowo przedstawione oszacowanie objętościowe i ustalić, czy w takiej konfiguracji różnice pomiędzy protokołami pozostają wykrywalne,
- wdrożenie systemu na platformie Kubernetes (AKS) z automatycznym skalowaniem horyzontalnym i badanie wpływu liczby replik na wydajność.

Bibliografia

- [1] Niswar M., Safruddin R.A., Bustamin A., Aswad I., *Performance Evaluation of Microservices Communication with REST, GraphQL, and gRPC*, International Journal of Electronics and Telecommunications, t. 70, nr 2, 2024, str. 429–436, DOI: <https://doi.org/10.24425/ijet.2024.149562>
- [2] Śliwa M., Pańczyk B., *Porównanie wydajności interfejsów programistycznych na przykładzie REST API, GraphQL i gRPC*, Journal of Computer Sciences Institute, t. 21, 2021, str. 356–361, DOI: <https://doi.org/10.35784/jcsi.2744>
- [3] Bolanowski M., Żak K., Paszkiewicz A., Ganzha M., Paprzycki M., Sowiński P., Lacalle I., Palau C.E., *Efficiency of REST and gRPC Realizing Communication Tasks in Microservice-Based Ecosystems*, w: *New Trends in Intelligent Software Methodologies, Tools and Techniques*, IOS Press, 2022, str. 97–108, DOI: <https://doi.org/10.3233/FAIA220242>
- [4] Kamiński Ł., Kozłowski M., Sporysz D., Wolska K., Zaniewski P., Roszczyk R., *Comparative Review of Selected Internet Communication Protocols*, Foundations of Computing and Decision Sciences, t. 48, nr 1, 2023, str. 39–56, DOI: <https://doi.org/10.2478/fcds-2023-0003>
- [5] Brito G., Mombach T., Valente M.T., *Migrating to GraphQL: A Practical Assessment*, IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2019, str. 140–150, DOI: <https://doi.org/10.1109/SANER.2019.8667986>
- [6] Vadlamani S.L., Emdon B., Arts J., Baysal O., *Can GraphQL Replace REST? A Study of Their Efficiency and Viability*, IEEE/ACM 8th International Workshop on Software Engineering Research and Industrial Practice (SER&IP), IEEE, 2021, str. 10–17, DOI: <https://doi.org/10.1109/SER-IP52554.2021.00009>
- [7] Jin R., Cordingly R., Zhao D., Lloyd W., *GraphQL vs. REST: Investigating Performance and Scalability for Serverless Data Persistence*, IEEE International Conference on Cloud Engineering (IC2E), IEEE, 2025, str. 155–161, DOI: <https://doi.org/10.1109/IC2E65552.2025.00031>
- [8] Newman S., *Building Microservices. Designing Fine-Grained Systems*, wyd. 2, O'Reilly Media, 2021.

- [9] Fowler M., Lewis J., Microservices – a definition of this new architectural term, <https://martinfowler.com/articles/microservices.html>, stan na dzień: 23.08.2026
- [10] Richardson C., *Microservices Patterns. With Examples in Java*, Manning Publications, 2018.
- [11] Evans E., *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley Professional, 2003.
- [12] Microsoft Corporation, .NET documentation, <https://learn.microsoft.com/pl-pl/dotnet/>, stan na dzień: 23.08.2026
- [13] TechEmpower, TechEmpower Framework Benchmarks, runda 23 (luty 2025), <https://www.techempower.com/benchmarks/>, stan na dzień: 23.08.2026
- [14] Microsoft Corporation, .NET and .NET Core Support Policy, <https://dotnet.microsoft.com/platform/support/policy/dotnet-core>, stan na dzień: 23.08.2026
- [15] Microsoft Corporation, Native AOT deployment, <https://learn.microsoft.com/dotnet/core/deploying/native-aot/>, stan na dzień: 23.08.2026
- [16] OpenTelemetry Authors, OpenTelemetry Documentation, <https://opentelemetry.io/docs/>, stan na dzień: 23.08.2026
- [17] Fielding R., *Architectural Styles and the Design of Network-based Software Architectures*, Doctoral Dissertation, University of California, Irvine, 2000.
- [18] Fowler M., Richardson Maturity Model, <https://martinfowler.com/articles/richardsonMaturityModel.html>, stan na dzień: 23.08.2026
- [19] Peon R., Ruellan H., HPACK: Header Compression for HTTP/2, RFC 7541, <https://www.rfc-editor.org/rfc/rfc7541>, stan na dzień: 23.08.2026
- [20] gRPC Documentation, <https://grpc.io/docs/>, stan na dzień: 23.08.2026
- [21] Google Developers, Protocol Buffers Documentation, <https://protobuf.dev/>, stan na dzień: 23.08.2026
- [22] gRPC-Web – gRPC for Web Clients, <https://grpc.io/docs/platforms/web/>, stan na dzień: 23.08.2026
- [23] Envoy Proxy Documentation, <https://www.envoyproxy.io/docs/>, stan na dzień: 23.08.2026

- [24] gRPC-Web Wire Format (PROTOCOL-WEB),
<https://github.com/grpc/grpc/blob/master/doc/PROTOCOL-WEB.md>, stan na dzień: 23.08.2026
- [25] gRPC-Web in ASP.NET Core gRPC apps, Microsoft Corporation,
<https://learn.microsoft.com/aspnet/core/grpc/grpcweb>, stan na dzień: 23.08.2026
- [26] Thomson M., Benfield C., HTTP/2, RFC 9113,
<https://www.rfc-editor.org/rfc/rfc9113>, stan na dzień: 23.08.2026
- [27] Google Developers, Protocol Buffers Reference Guides,
<https://protobuf.dev/reference/>, stan na dzień: 23.08.2026
- [28] Microsoft Corporation, APIs overview – Minimal APIs in ASP.NET Core,
<https://learn.microsoft.com/aspnet/core/fundamentals/minimal-apis/overview>,
stan na dzień: 23.08.2026
- [29] Microsoft Corporation, Cache-Aside pattern, Azure Architecture Center,
<https://learn.microsoft.com/azure/architecture/patterns/cache-aside>, stan na
dzień: 23.08.2026
- [30] Microsoft Corporation, Playwright Documentation,
<https://playwright.dev/docs/intro>, stan na dzień: 23.08.2026
- [31] Buf Technologies, Connect for Web Documentation,
<https://connectrpc.com/docs/web/getting-started/>, stan na dzień: 23.08.2026
- [32] Google LLC, Chrome DevTools Protocol,
<https://chromedevtools.github.io/devtools-protocol/>, stan na dzień: 23.08.2026
- [33] Eddy W., Transmission Control Protocol (TCP), RFC 9293,
<https://www.rfc-editor.org/rfc/rfc9293>, stan na dzień: 23.08.2026
- [34] Jain R., *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*, John Wiley & Sons, 1991.

Spis rysunków

6.1	Pulpit Grafany: liczba żądań na sekundę oraz percentyle czasu obsługi żądania po stronie usług <code>ProductService</code> i <code>OrderService</code>	33
6.2	Pulpit Grafany: liczba żądań w podziale na trasy oraz metryki środowiska uruchomieniowego .NET, wolumen alokacji i częstość odświeżania pamięci	34
7.1	Konsola Hetzner Cloud: parametry serwera, na którym wdrożono system testowany	42
7.2	Wykorzystanie procesora oraz przepustowość dysku serwera w trakcie pracy systemu pod obciążeniem (pulpit konsoli Hetzner Cloud) .	42
8.1	Mediana opóźnień w scenariuszach Echo względem liczby rekordów w odpowiedzi	53
8.2	Mediana opóźnień w scenariuszach Products z ciepłą pamięcią podręczną, dla dziesięciu i dwudziestu równoległych klientów; słupki błędów odpowiadają 95% przedziałowi ufności wartości centralnej. . .	55
8.3	Mediana opóźnień w scenariuszach Products z zimną pamięcią podręczną; słupki błędów odpowiadają 95% przedziałowi ufności wartości centralnej	56
8.4	Mediana opóźnień w scenariuszach Orders względem liczby pozycji zamówienia; słupki błędów odpowiadają 95% przedziałowi ufności wartości centralnej	57
8.5	Różnice czasów trzech konfiguracji względem mediany opóźnienia REST; wartości procentowe odniesiono do mediany REST	58
8.6	Opóźnienie i przepustowość względem liczby równoległych klientów; szary pas oznacza wspólny pułap przepustowości, a różowy położenie punktu przegięcia	59

Spis tabel

1.1 Zakres omówionych badań i ograniczenia ich odniesienia do eksperymentu przeglądarkowego	12
2.1 Porównanie architektury monolitycznej i mikroserwisowej; opracowanie własne na podstawie [8, 10]	17
4.1 Porównanie natywnego gRPC, gRPC-Web oraz REST w kontekście komunikacji frontend-backend	24
6.1 Limity zasobów kontenerów w konfiguracji wdrożeniowej systemu autorskiego	31
7.1 Wymiary macierzy badawczej	41
7.2 Zmierzona charakterystyka łącza klient–serwer	43
7.3 Zweryfikowana warstwa transportowa poszczególnych ścieżek komunikacji	44
8.1 Zmierzony rozmiar odpowiedzi [B] i redukcja uzyskana serializacją binarną	49
8.2 Rozmiar ładunku scenariusza Echo bez kompresji i po kompresji gzip [B] oraz wynikająca z niego liczba okien odbiorczych	50
8.3 Weryfikacja modelu: przewidywana i zmierzona liczba dodatkowych obiegów pakietu	51
8.4 Mediana opóźnienia w scenariuszach Echo [ms]; w nawiasach współczynnik zmienności median pięciu przebiegów	52
8.5 Mediana opóźnienia w scenariuszach Products, ciepła pamięć podręczna [ms]; w nawiasach współczynnik zmienności median pięciu przebiegów	54
8.6 Mediana opóźnienia w scenariuszach Products, zimna pamięć podręczna [ms]; w nawiasach współczynnik zmienności.	54
8.7 Mediana opóźnienia w scenariuszach Orders [ms]; w nawiasach współczynnik zmienności	56
8.8 Czasy konfiguracji i różnice względem mediany REST	57

8.9 Skalowanie z liczbą równoległych klientów: mediana opóźnienia [ms] i przepustowość [operacji/s]	58
8.10 Współczynnik zmienności median pięciu przebiegów, w podziale na grupy scenariuszy	59
8.11 Przedziały ufności 95% dla wartości centralnej opóźnienia [ms]; $k = 5$ przebiegów, $t = 2,776$	60
8.12 Przedziały ufności 95% dla różnic sparowanych w obrębie przebiegu [ms]; wartość dodatnia oznacza dłuższy czas odpowiedzi pierwszego z protokołów	61

Spis listingów

3.1	Definicja punktu końcowego REST z wykorzystaniem Minimal API w .NET 10	20
3.2	Definicja serwisu gRPC w formacie Protocol Buffers	21
4.1	Generowanie kodu klienckiego dla Connect RPC	26
7.1	Punkt końcowy Echo izolujący narzut protokołu od warstwy danych .	36

Streszczenie

Praca pt. „*Architektura mikroserwisów i rozwiązania chmurowe w ekosystemie .NET*” poświęcona jest projektowaniu, implementacji oraz empirycznej analizie wydajności systemu rozproszonego zbudowanego w oparciu o platformę .NET 10. W ostatnim czasie obserwuje się intensywny rozwój protokołu gRPC jako alternatywy dla powszechnie stosowanego REST w komunikacji między mikroserwisami. Jednocześnie środowisko przeglądarek internetowych wymaga stosowania wariantu *gRPC-Web*, którego wydajność w komunikacji klient–backend pozostaje słabo zbadana empirycznie. W rozważanej pracy podjęto zatem próbę wypełnienia tej luki badawczej. W szczególności zebrano wiadomości z zakresu architektury mikroserwisów, mechanizmów komunikacji synchronicznej i asynchronicznej oraz wzorców projektowych w ekosystemie .NET, porównano trzy warianty komunikacji dostępne dla aplikacji przeglądarkowej: REST (HTTP/1.1 z serializacją JSON), gRPC-Web przez proxy Envoy oraz gRPC-Web obsługiwany bezpośrednio w ASP.NET Core, zrealizowano system testowy złożony z dwóch mikroserwisów (*ProductService* i *OrderService*), wdrożony na serwerze chmury Hetzner, a także opracowano metodykę pomiarową opartą na rzeczywistej przeglądarce internetowej, w której czas mierzony jest od interakcji użytkownika do naniesienia odpowiedzi na drzewo dokumentu. Przeprowadzono 39 konfiguracji pomiarowych w pięciu niezależnych powtórzeniach z rotowaną kolejnością protokołów, co dało 585 wykonanych testów i 171 tysięcy pomiarów, przy rozmiarach ładunku od 10 do 2000 rekordów oraz scenariuszach obejmujących izolowany narzut protokołu, odczyt katalogu i zapis zamówienia. Zaprojektowany i zaimplementowany system charakteryzuje się następującymi cechami: podejściem *contract-first* opartym na *Protocol Buffers* zapewniającym silną typizację w całym stosie technologicznym oraz kompleksową obserwowalnością zgodną ze standardem OpenTelemetry, obejmującą metryki eksportowane do Prometheusa i wizualizowane w Grafanie. Przeprowadzone badania wykazały, że warianty gRPC-Web skracają czas odpowiedzi o 26–43% względem REST, lecz wyłącznie przy nieskompresowanych odpowiedziach przekraczających rozmiar okna odbiorczego protokołu TCP; przy małych ładunkach różnice były niewielkie, a w operacjach zapisu nie wykryto stałej przewagi jednego wariantu. Brak rozstrzygnięcia różnic w zapisie nie dowodzi równoważności protokołów. Wykazano ponadto, że kompresja treści redukuje

wolumen dokumentu JSON dwunasto- do trzynastokrotnie i sprowadza wszystkie badane ładunki poniżej rozmiaru jednego okna odbiorczego, co wyznacza granicę stosowalności powyższego wniosku. Sformułowano model wyjaśniający tę zależność liczbą wymian pakietów wynikającą z wolumenu odpowiedzi, którego przewidywania zgadzają się z pomiarem z dokładnością do 0,29 obiegu pakietu. Wykazano ponadto, że różnica REST–Envoy stanowi około 85–98% całkowitej różnicy REST–Direct w pięciu konfiguracjach dekompozycji, co wspiera znaczenie redukcji wolumenu danych. Porównanie kompletnych ścieżek daje przybliżony podział kosztów. Model przewiduje zależność korzyści od opóźnienia sieci.

Summary

The thesis entitled “*Microservices Architecture and Cloud Solutions in the .NET Ecosystem*” is devoted to the design, implementation, and empirical performance analysis of a distributed system built on the .NET 10 platform. Recently, rapid growth and adoption of the gRPC protocol have been observed as an alternative to the widely used REST for inter-microservice communication. At the same time, the web browser environment necessitates the use of the *gRPC-Web* variant, whose performance in client–backend communication remains insufficiently investigated empirically. This study therefore aims to address and fill this research gap. In particular, the thesis synthesizes knowledge in microservices architecture, synchronous and asynchronous communication mechanisms, and design patterns within the .NET ecosystem; compares three communication variants available to web browser applications: REST (HTTP/1.1 with JSON serialization), gRPC-Web via an Envoy proxy, and gRPC-Web natively hosted in ASP.NET Core; implements a benchmark testbed consisting of two microservices (*ProductService* and *OrderService*) deployed on a Hetzner Cloud server; and establishes a measurement methodology based on a real web browser, where end-to-end latency is measured from the initial user interaction to the rendering of the response into the Document Object Model (DOM) tree. A total of 39 measurement configurations were executed across five independent repetitions with rotated protocol ordering, yielding 585 test runs and 171 thousand individual measurements, spanning payload sizes from 10 to 2000 records across scenarios covering isolated protocol overhead, catalog read, and order placement. The designed and implemented system is characterized by a *contract-first* approach based on *Protocol Buffers* providing strong typing across the entire technology stack and comprehensive observability compliant with the OpenTelemetry standard, encompassing metrics exported to Prometheus and visualized in Grafana. The empirical evaluation revealed that gRPC-Web variants reduce response times by 26–43% compared to REST, but exclusively for uncompressed responses exceeding the initial TCP receive window size; for small payloads, differences were small, while write operations showed no consistent advantage for one variant. Inconclusive differences in write operations do not establish protocol equivalence. Furthermore, it was demonstrated that response compression reduces JSON document size by a factor of 12–13, bringing all evaluated payloads below the threshold of a single receive

window, thereby defining the boundary of practical applicability for the aforementioned performance advantage. A model accounting for this relationship based on the number of network round-trips dictated by payload volume was formulated, with predictions aligning with empirical measurements within 0.29 packet round-trips. In addition, it was established that the REST–Envoy difference accounts for approximately 85–98% of the total REST–Direct difference in the five decomposition configurations, supporting the importance of payload reduction. This comparison of complete request paths provides an approximate cost breakdown. The model predicts that the benefit depends on network latency.

Słowa kluczowe

mikroserwisy;
gRPC-Web;
REST API;
Protocol Buffers;
ASP.NET Core;
Envoy Proxy;
testy wydajnościowe;
architektura rozproszona;
.NET 10;
HTTP/2;